

SW 품질을 위한 Agentic Engineering

Agenda

[SW 품질을 위한 Agentic Engineering]

- I. Agent, 공학적으로 바라보기
- II. Agentic Engineering 소개
- III. Phase별 App 개발 실습 (Pang KATA)
- IV. Coding Agent의 Engineering 발전 과정
- V. 비용 관점으로 바라본 Agentic Engineering

[Agentic Engineering 활용 전략]

- VI. Agentic Engineering을 적용한 유지보수
- VII. 좋은 코드 판정하기
- VIII. 리팩토링 실습 (차량조립 KATA)
- IX. PoC와 Collection 수집
- X. AE 관점으로 바라보기 (with SubAgent)

교육 내용 및 학습 목표

- Agentic Engineering 개념을 익히고, Vibe Coding과 차이점을 이해한다.
- AI와 사람의 역할을 분리하는 실습을 진행한다. (Pang KATA)
- 이후 Coding Agent의 발전과정과 ROI 관점에서 Agentic Engineering을 살펴본다.

Chapter 01

Agent, 공학적으로 바라보기

SW 품질을 위한 Agentic Engineering

공학적인 관점의 전환

Agent를 비판적 사고로 바라보자

- 지금부터 Coding Agent에서 다뤄지는 내용을 공학적 관점으로 접근해본다.
- 공학적 관점에 접근하는 가장 좋은 방법은 **비판적 사고**이다.

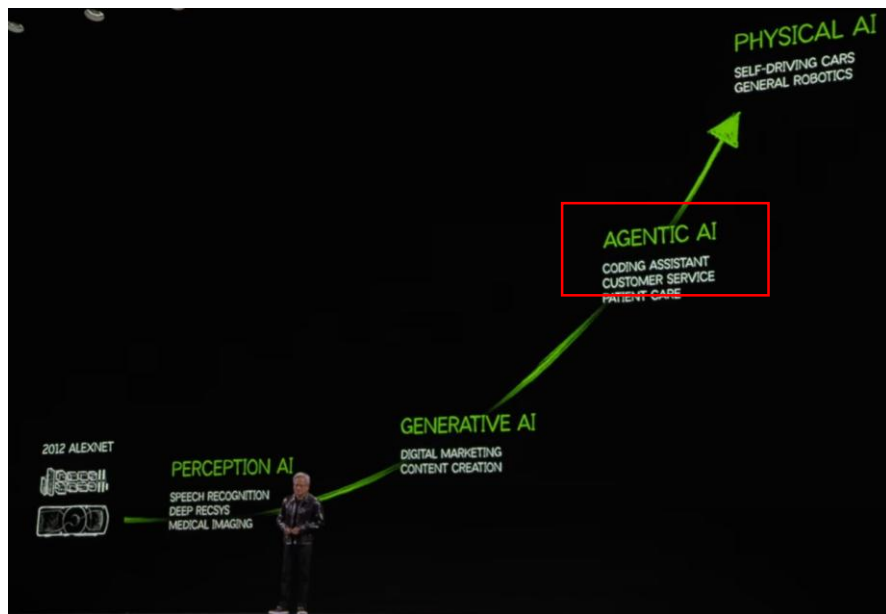
Agentic Engineering은 SW품질을 위해 반드시 필요한 접근 방식이다.
하지만, 그 과정 하나하나를 무조건 수용하기 보다는 비판적으로 바라보자.
이렇게 비판적으로 검토했을 때에도, 충분히 납득이 된다면 받아들이고 실무에 적용하자.

Agent 용어 부터 이해하기

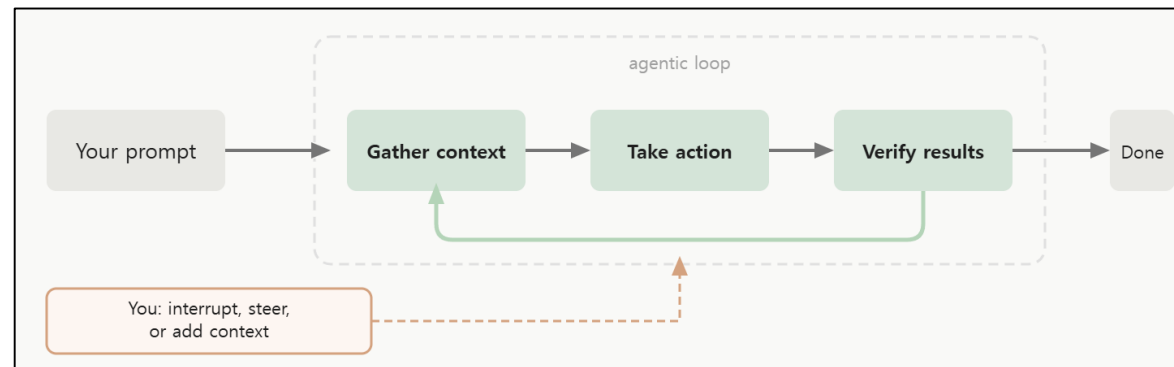
Agent란 = 특정 목적을 달성하도록 설계된 프로그램

- LLM을 통해 추론을 수행하고, 그 결과에 따라 **도구를 실행**함
- 실행 결과를 바탕으로 다시 LLM이 판단하고, 필요 시 추가 작업을 반복함
- 이러한 과정을 통해 목표 달성 여부를 스스로 점검함

에이전트는 목표를 달성하기 위해 도구를 **반복적으로** 실행하는 구조를 가진다.



NVIDIA CEO 젠슨황이 CES 2025 키노트에서 꼭지점을 언급



(그림 출처 : <https://code.claude.com/docs/ko/how-claude-code-works>)

Coding Agent

(코드 작성 / 실행 / 테스트) 반복

코딩 에이전트는 특정 목표를 달성하기 위해 코드 작성과 실행, 테스트를 반복하는 프로그램이다.

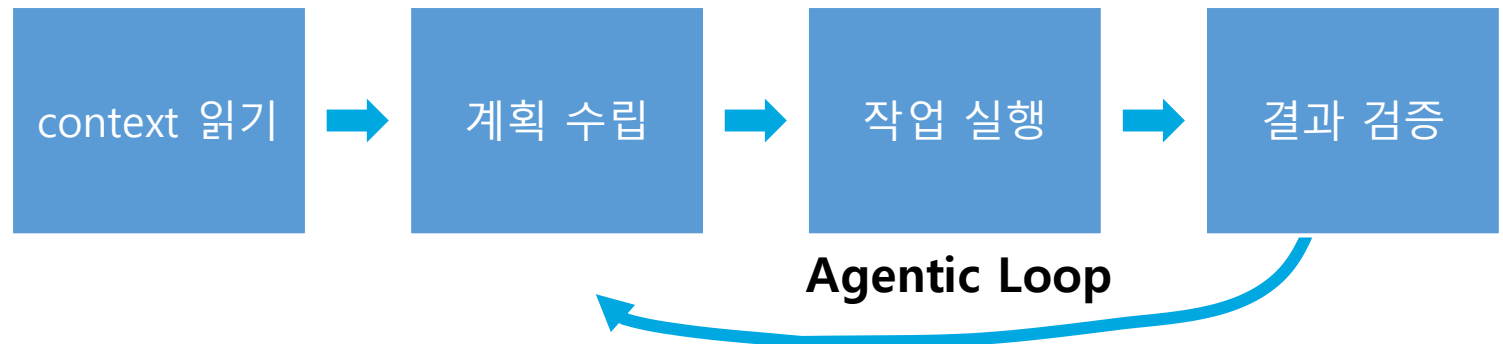
→ 단순히 코드를 생성하는 데서 끝나지 않고,
실행 결과를 바탕으로 스스로 수정하고 개선해 나간다.

종류

Claude Code (Market Leader)

Codex

Cursor



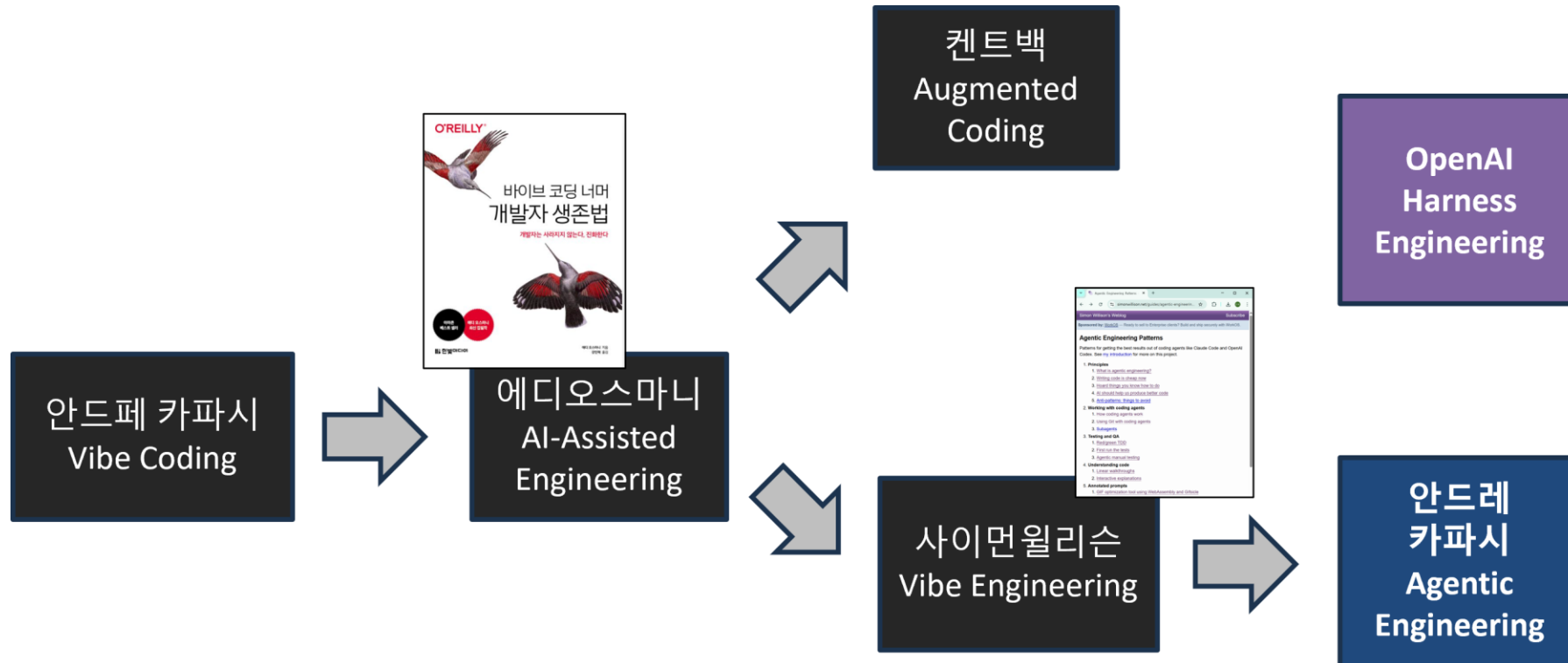
문제점 : 무지성 Coding Agent를 사용

- '딸깍' 으로 SW 품질이 쉽게 무너지는 현상 발생
- 이러한 문제점들을 해결하는 공학적 접근이 필요해졌다.

이에 따라 공학적 이론을 기반으로 접근하여,
SW 품질을 확보하면서도 생산성을 동시에 달성하려는 다양한 방법론들이 등장하기
시작했다.

Agentic Engineering 방법론 등장

에디오스마니, 사이먼월리슨, 안드레카파시 등을 거쳐 방법론이 자리를 잡아갔다.
Agent를 활용하는데 공학적 접근방법(엔지니어링)이 필요하다 라는 의미에서
Agentic Engineering으로 이름이 정해졌다. (by 안드레카파시)



[참고] 출처

애디오스마니

- <https://addyosmani.com/blog/agent-engineering/>

사이몬윌리슨

- <https://simonwillison.net/2025/Oct/7/vibe-engineering/>
- <https://simonwillison.net/guides/agent-engineering-patterns/>

켄트백

- <https://tidyfirst.substack.com/p/augmented-coding-beyond-the-vibes>

Chapter 02

Agentic Engineering 소개

SW 품질을 위한 Agentic Engineering

Agentic Engineering 의 시작점

Agentic Engineering 방법론은 **AI와 사람의 역할을 명확히 구분하는 것으로 시작된다.**

- AI의 역할: 설계 도움, 초안 작성, 코드 구현
- 사람의 역할: 설계, 기획, AI가 만든 작업물에 대한 리뷰와 승인

→ 두 역할의 구분이 SW 품질을 높일 수 있는 Agentic Engineering 의 핵심이다.



AI를 사용하는 SW 엔지니어들의 미션

어떤 코드를 작성해야 하는지에 대해 명확하게 고민을 해왔다.

1. LLM에게 어떻게 입력을 잘 줄 것인가에 대한 고민 (Prompt Engineering)
2. LLM에게 주는 입력을 어떻게 체계화 할 것인가에 대한 고민 (Context Engineering)
3. Coding Agent를 활용해 안정적으로 SW를 생성해 내기 위한 **환경을 어떻게 설계할 것인지** 고민 (Harness Engineering)
 - 문제 해결에 필요한 적절한 도구를 제공한다.
4. Coding Agent와 함께 **최고 수준의 고품질의 SW**를 만들기 위해 어떻게 **협업할지** 고민 (Agentic Engineering)
 - 문제를 적절한 수준으로 구체화하고, 세분화한다. (문제 분해 능력)
 - 생성된 결과를 반드시 사람이 검증한다.

“바이브코딩” vs Agentic Engineering

바이브코딩이란, 코드가 존재한다는 사실을 잊어버리는 방식이라고 안드레카파시가 정의함
이 정의는 계속 가져가면서, Agentic Engineering은 별개의 개념으로 가져가야 한다. (by 사이먼 윌리슨)

바이브코딩은 LLM을 사용하여 코드를 생성하는 모든 경우를 뜻하는 용어가 아니다.

- ‘코드 존재를 잊어버린다’ 라는 것에 집중한 용어이다.

Agentic Engineering은 바이브 코딩과 다르게 어떠한 코드가 만들어졌는지 AI와 리뷰를 해야 한다.

바이브 코딩
으로 만든 코드



Agentic Engineering
으로 만든 코드

“바이브코딩” vs Agentic Engineering

바이브코딩이란, 코드가 존재한다는 사실을 잊어버리는 방식이라고 안드레카파시가 정의함
이 정의는 계속 가져가면서, Agentic Engineering은 별개의 개념으로 가져가야 한다. (by 사이먼 윌리슨)

바이브코딩은 LLM을 사용하여 코드를 생성하는 모든 경우를 뜻하는 용어가 아니다.

- ‘코드 존재를 잊어버린다’ 라는 것에 집중한 용어이다.

Agentic Engineering은 바이브 코딩과 다르게 어떠한 코드가 만들어졌는지 AI와 리뷰를 해야 한다.

바이브 코딩
으로 만든 코드



Agentic Engineering
으로 만든 코드

Agentic Engineering은 Model이 발전해도 구식이 되는 이론이 아니다.

Agentic Engineering은 일시적인 기술 트렌드가 아니라,
AI와 인간의 역할을 구조적으로 분리하는 공학적 접근 방식이기에
시간이 지나도 구식이 되는 이론이 아니다.

모델의 성능이 아무리 발전하더라도,
사람이 직접 품질과 방향을 통제하며,
최종 책임까지 지는 방향성은 모델이 아무리 발전해도 변하지 않는다.

지금부터 Agentic Engineering 교육은 역할 분리에 집중한다.

사람이 할 일과 AI가 할 일을 명확히 구분하고, 사람은 자신의 역할에 집중하여 진행한다.

고품질의 소프트웨어를 만들기 위해,
어떤 일을 사람이 해야 하는지,
어떤 일을 AI가 담당해야 하는지를 구분하며 실습에 참여하자.

Chapter 03

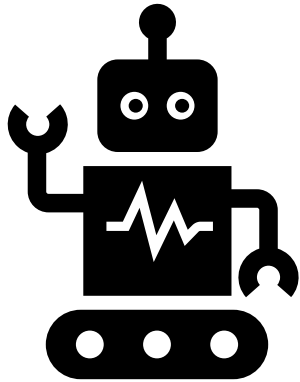
Phase별 App 개발 실습 (Pang KATA)

SW 품질을 위한 Agentic Engineering

챕터의 목적

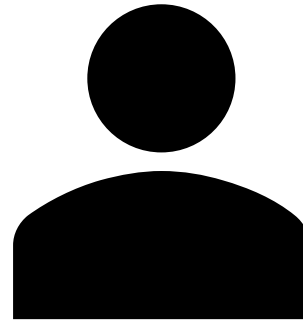
본격적으로 Agentic Engineering을 학습하기 앞서
바이브 코딩 실습을 통해, AI와 사람의 역할을 분리하는 경험을 갖는다.

이 경험은 Agentic Engineering에 대한 고민을 하는데 발판이 된다.



Coding Agent

초안작성
구현 담당



사람

목표설정
검토 담당

목표

Pang 게임 개발 및 검토 경험

AI 협업 문서 관리 (Context Engineering 경험)

AI 산출물에 대한 검토

실습은 Web App으로 진행하나, 코드에 대한 리뷰는 진행하지 않음
(기술 스택 : React.js, Vite, TypeScript)



Bubble 모두 터트리는 아케이드 게임

맵에서 튕겨다니는 모든 Bubble을 전부 제거 하는 것이 목적인 게임

원본 게임 영상 : <https://www.youtube.com/shorts/lXZV3DX8X2w>

4개의 Object로 구성

1. Player (주인공, 좌우로만 이동 가능)
2. Wire (수직으로 위로만 총 발사 가능)
3. Bubble (제거해야 하는 방울)
4. Block (벽)



Pang KATA 규칙

실제 게임보다 단순화된 규칙을 갖는다.



Player

키보드 방향키 좌우 버튼으로 이동
방울에 닿으면 목숨을 잃는다.
수직 방향 위로만 Wire 발사 가능
모든 Bubble을 소멸시키면 승리



Bubble

버블은 좌/우 방향으로 공처럼
튀겨진다.
중력의 영향을 받고, 튕길 때마다
같은 높이까지 올라간다.
작은 버블은 Wire를 맞으면 소멸된다.
화면 밖으로 Bubble이 나갈 수 없다.

Wire



Space키로 Wire 발사
Bubble을 맞추면 작은 2개로
분할된다.
벽은 뚫지 못한다.

Block



벽은 부서지지 않는다.
Bubble은 Block 위로 튕겨질 수 있다.

Github - 제출용 Remote Repository 생성

Github 에 Remote Repository를 생성

1. Owner : 실습용 Organization 선택

2. Repository 이름

repo 이름 : PANG-랜덤한 번호

Description : 소속 및 이름

Create a new repository

A repository contains all project files, including the revision history.

Owner: reviewer-01 / Repository name: PANG-번호

Great repository names are short and memorable. Need inspiration? How about redesigned-journey?

Description (optional): 팀원 이름

☐ Public
Any logged in user can see this repository. You choose who can commit.

☐ Internal
Samsung SDS enterprise members can see this repository. You choose who can commit.

☒ Private
You choose who can see and commit to this repository.

Skip this step if you're importing an existing repository.

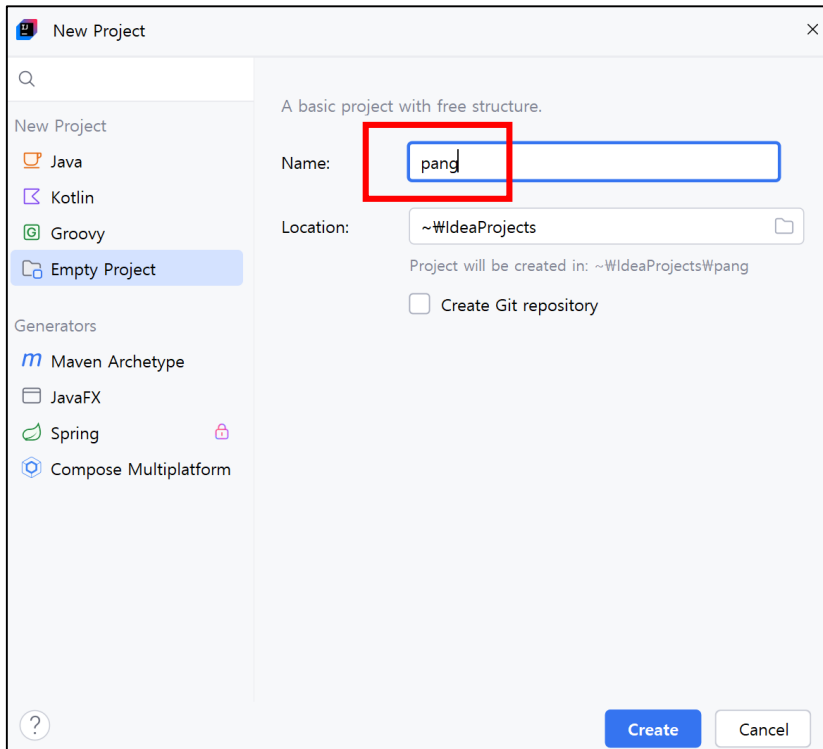
☐ Initialize this repository with a README
This will let you immediately clone the repository to your computer.

Add .gitignore: None

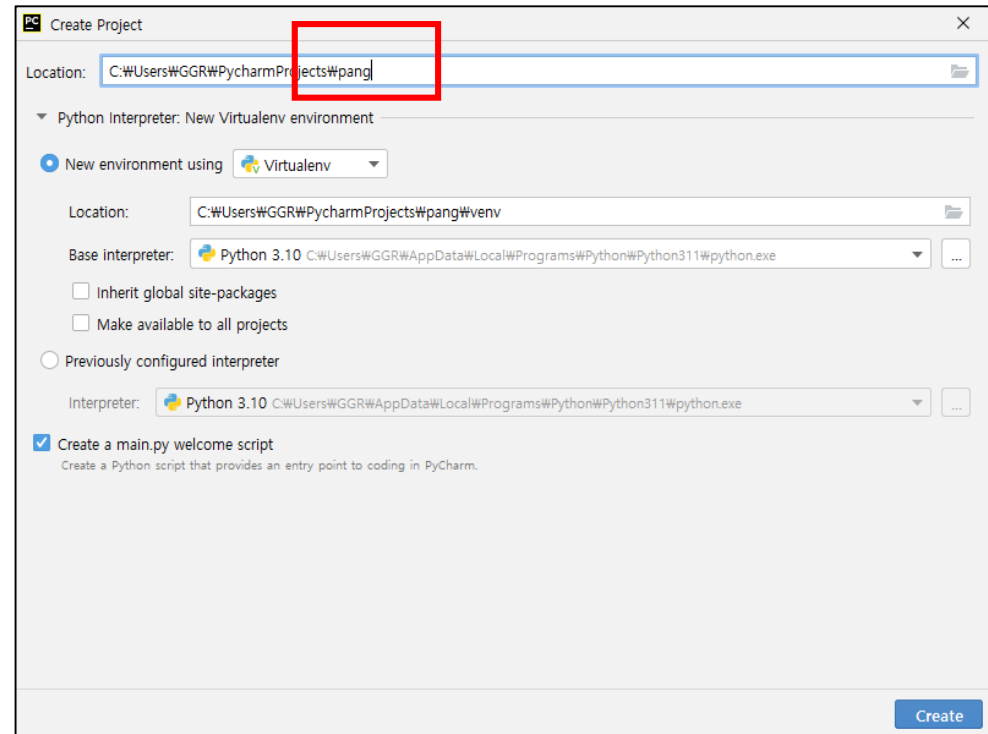
Create repository

프로젝트 생성 – Java / Python

비어 있는 프로젝트 생성
프로젝트 이름 : pang



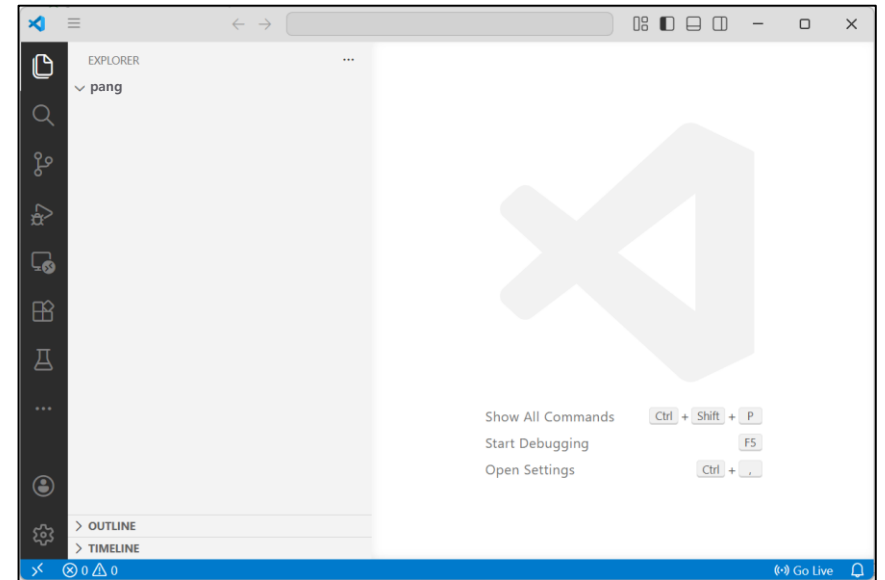
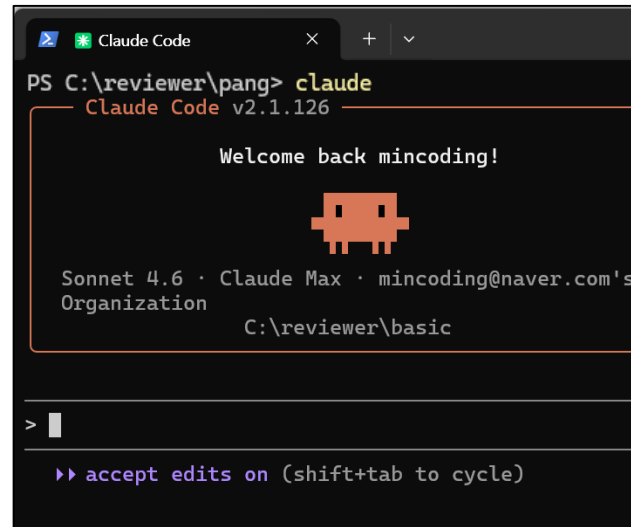
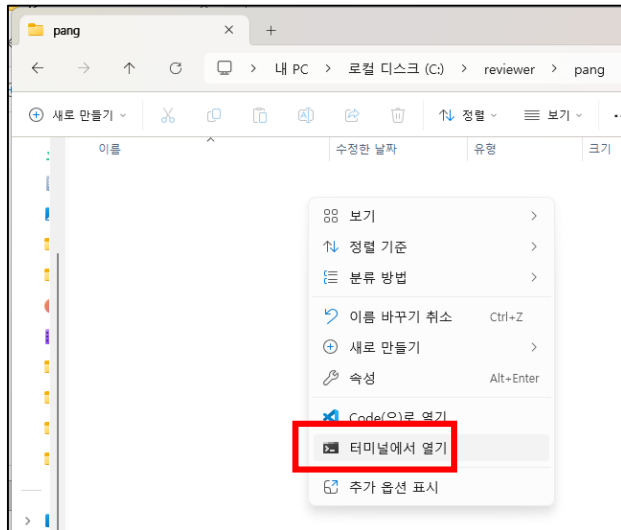
Java(IntelliJ) 활용 시



Python(Pycharm) 활용 시

프로젝트 생성 - c++

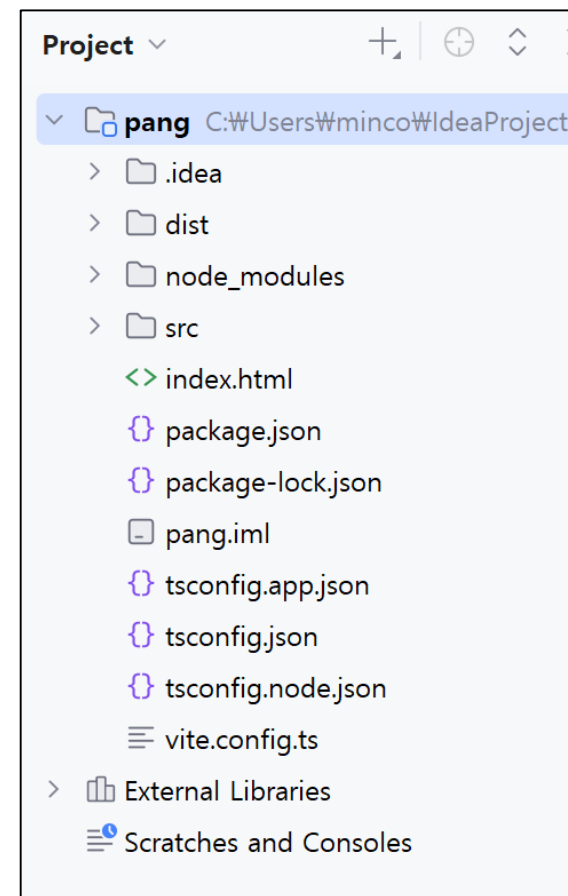
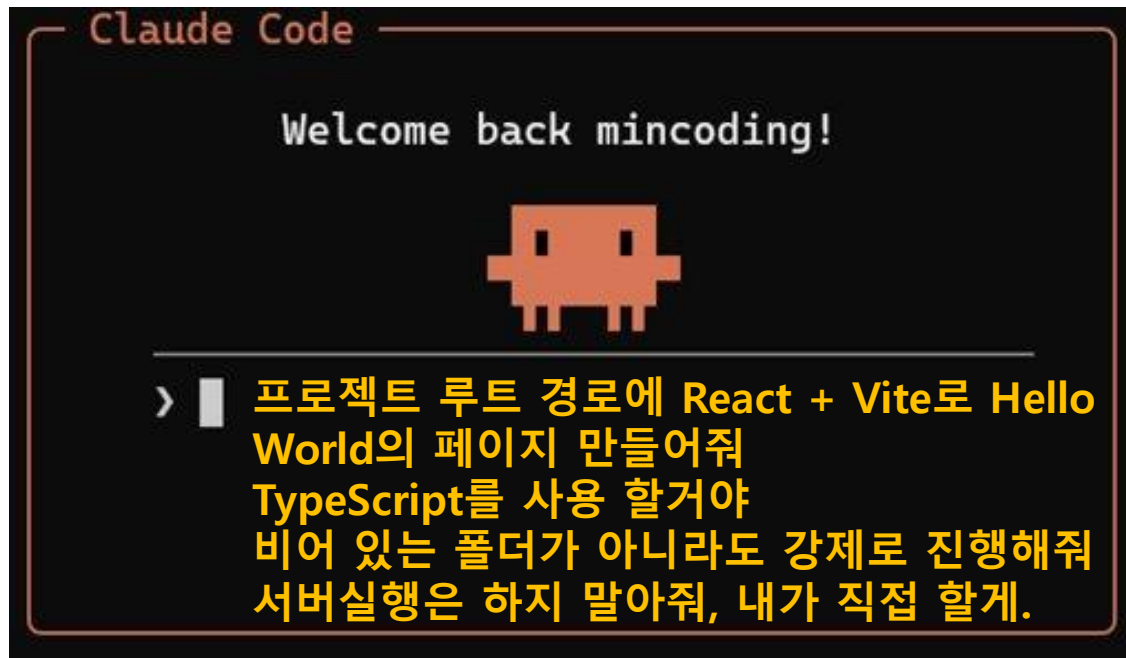
- 1) PowerShell + Claude Code로 CLI 기반으로 사용
- 2) vscode (Visual Studio Code)를 “문서에디터” 용도로만 사용
 - 프로그래밍 용도가 아닌, 문서 Editor 용도로만 사용할 예정
 - 설치 : <https://code.visualstudio.com/>
C:\reviewer\wpang 폴더 생성 후 진행



C++(Powershell + Claude code) 활용 시

프로젝트 생성

Hello World 프로젝트 생성



프로젝트 생성

제작한 Hello World 프로젝트를
실행하는 방법을 물어본 후,
직접 Terminal 창에서 실행

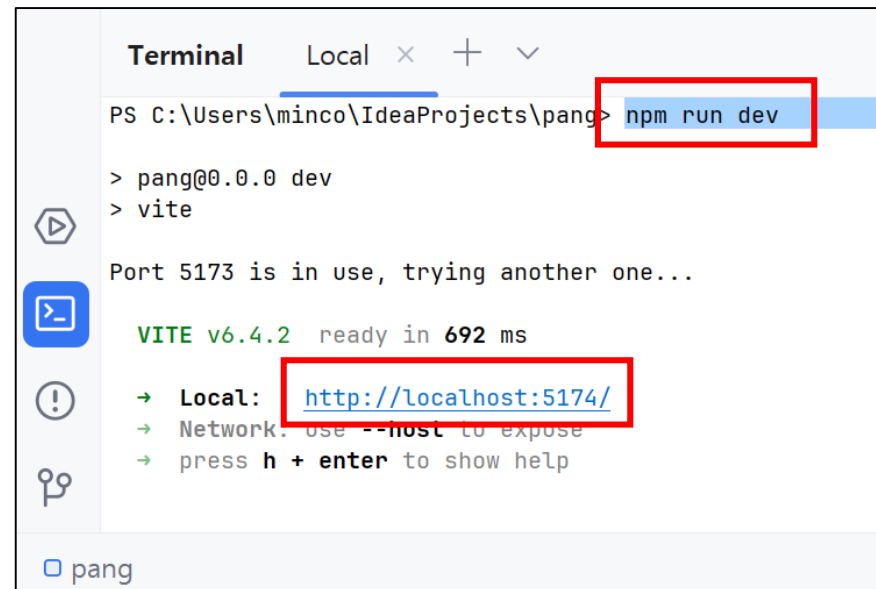


서버 실행 방법 알려줘

- 터미널에서 프로젝트 디렉토리로 이동 후 아래 명령을 실행하세요 :

```
cd C:\Users\minco\IdeaProjects\pang  
npm run dev
```

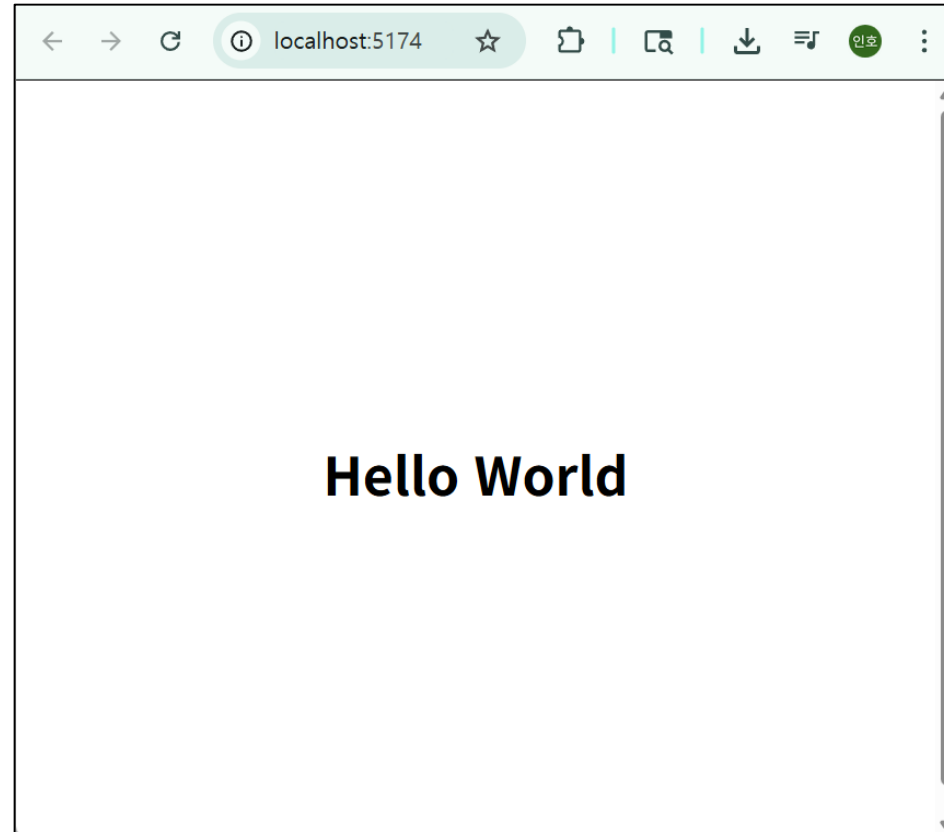
브라우저에서 <http://localhost:5173> 으로 접속하면 됩니다.



Coding Agent는 랜덤성에 의해 동작하므로, 실습 결과가 이미지와 다를 수 있습니다.
또한, 위 실습 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

프로젝트 생성

npm run dev를 통해 실행된 Web App 동작 확인

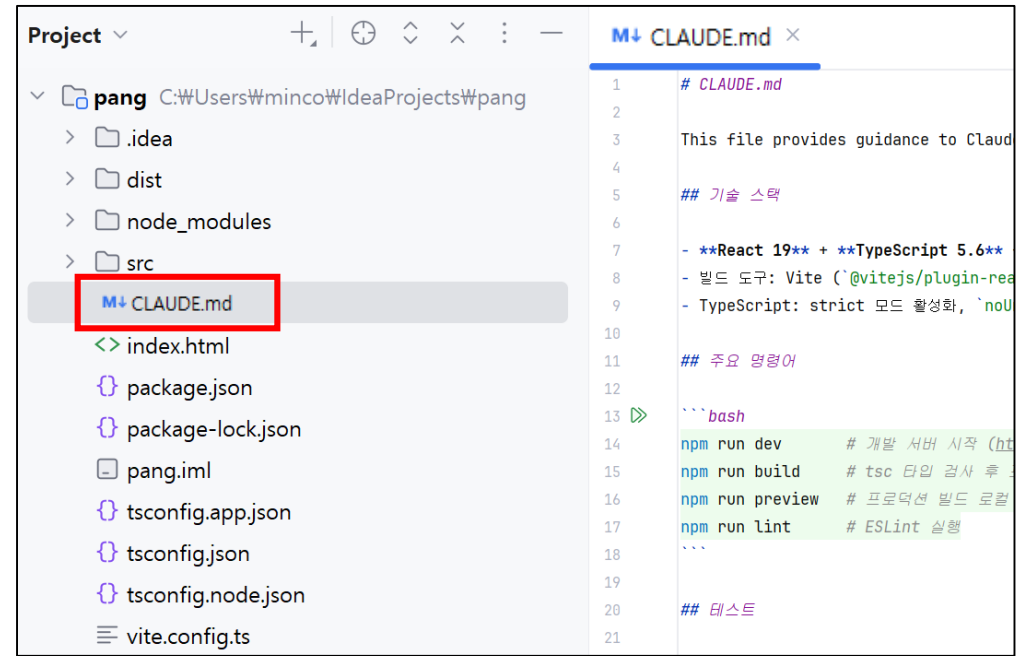


Coding Agent는 랜덤성에 의해 동작하므로, 실습 결과가 이미지와 다를 수 있습니다.
또한, 위 실습 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

CLAUDE.md 문서 생성

/init skill을 사용하여 CLAUDE.md 문서 생성
기술 스택 + 테스트 방법 명시

현재 프로젝트 소스코드를 읽고,
CLAUDE.md를 생성해주는 Skill



[도전] CLAUDE.md 에 불필요한 내용 삭제하기

만들어진 CLAUDE.md 를 사람이 직접 검토한다.

CLAUDE.md는 Agent용 문서이지만,
사람이 관리를 해야 하는 문서이다.

좋은 CLAUDE.md에 대한 고민을 해볼 수 있는 글

<https://addyosmani.com/blog/agents-md/>

한글요약 : <https://news.hada.io/topic?id=26972>

좋지 않은 내용이 있다면 과감히 삭제한다.

- 1) 다른 문서에도 있는 중복적인 내용
- 2) 코드를 통해 쉽게 파악할 수 있는 내용
- 3) 디렉토리 구조 등 변경이 잦은 내용
- 4) 당연한 내용

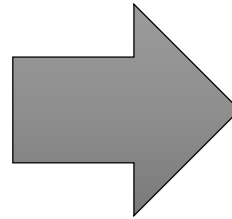
"이 규칙이 3개월 뒤에도 유효한가? --> YES면 CLAUDE.md

```
M↓ CLAUDE.md ×
1  # CLAUDE.md
2
3  This file provides guidance to Claude Code (claude.ai/code) when working with this repository.
4
5  ## 기술 스택
6
7  - **React 19** + **TypeScript 5.6** + **Vite 6**
8  - 빌드 도구: Vite (@vitejs/plugin-react - Babel 기반 Fast Refresh)
9  - TypeScript: strict 모드 활성화, `noUnusedLocals` / `noUnusedParameters`
10
11 ## 주요 명령어
12
13 >> ```bash
14 npm run dev      # 개발 서버 시작 (http://localhost:5173)
15 npm run build    # tsc 타입 검사 후 프로덕션 빌드 (dist/)
16 npm run preview  # 프로덕션 빌드 로컬 미리보기
17 npm run lint     # ESLint 실행
18 ```
19
20 ## 테스트
21
22 현재 테스트 프레임워크가 설정되어 있지 않습니다. 테스트를 추가할 경우 Vitest 사용
23
24 >> ```bash
25 # Vitest 추가 시
26 npm install -D vitest @testing-library/react @testing-library/user-event
27 # package.json scripts에 "test": "vitest" 추가
28 npm run test     # 전체 테스트 실행
29 npm run test -- 파일명 # 단일 파일 테스트 실행
30 ```
```

초기 설정 commit

- 프로젝트를 진행하기 위한 초기 설정이 완료됨에 따라 commit으로 이력 기록
- commit을 남겨야 AI로 작업하다 이상이 생겼을 경우 되돌아오기 용이함

여기까지가 프로젝트 구현을 위한 준비였고, 이를 commit으로 기록해



claude

초기 commit 이 완료 완료되었습니다. 되었습니다. 20 개의20 개의 파일 (AG 파일 (AGENTS.md, 프로젝트 설정ENTS.md, 프로젝트 설정 파일 파일, 소스, 소스 코드 등) 코드 등) 이 commit 이 commit 되었습니다. 되었습니다.

✓ Created by generative AI in 15s

terminal

```
code56@SDS-DEDU-50006 MINGW64 /c/reviewer/workspace/Pang (master)
$ git log
commit bbb985b910af17ad7d12e306af3eee7d933c45db (HEAD -> master)
Author: unknown <code56@smartdesk.sds.cloud.com>
Date: Tue Apr 28 14:15:32 2026 +0900
```

Initial commit: Pang React TypeScript project setup

사용자 입장에서 필요한 기능에 대한 요구사항을 기입한 문서

- 기술적인 내용을 다루지 않음
- 전체적인 요구사항에 대한 문서
 - 부분적으로 디테일한 요구사항은 FEATURE Directory를 하나 만들어서 관리

관행적으로 많이 사용하는 구조

1. docs/
├ 01_PRD.md
└ 02_SPEC.md

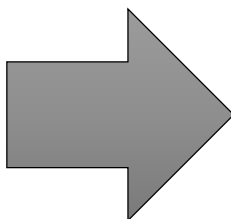
파일명, 구조 등 자유롭게 변형하여 사용 가능

2. docs/
├ PRD.md
├ FEATURES/
│ ├── auth.md / 로그인/회원가입 상세
│ ├── post.md / 게시글 기능 상세
│ └── comment.md / 콘텐츠 상세

PANG 게임에 대한 PRD 초안 작성을 지시

디테일한 요구사항이 아닌 전체적인 요구사항 기록

팡(PANG) 게임을 만들꺼야
팡 게임 Mission 1에 대해 일단 너가 검색하고
그 특징을 docs/PRD.md 파일에 작성해
디테일한 내용보다는 전체적인 내용으로 작성해봐



```
# Product Requirements Document (PRD)

## 팡 (PANG) 게임

### Overview

팡 (PANG) 은 고전적인 버블 팝 (Bubble Pop) 게임으로, 플레이어가 발

### 게임 컨셉

- **핵심 메커니즘**: 하단에서 공을 발사하여 풍선을 터뜨림
- **풍선 분할**: 큰 풍선은 터질 때 작은 풍선 3 개로 분할됨
- **목표**: 제한된 시도 내에 모든 풍선을 제거
- **장애물**: 회전하는 막대 (바리케이드) 를 피해야 함

### Mission 1 특징

**초보자용 튜토리얼 레벨**

- **풍선 수**: 5 개 (기본 풍선)
- **난이도**: 매우 낮음
- **막대**: 없음 또는 매우 느린 회전 속도
- **목표**: 게임 기본 조작법 학습
  - 마우스/터치로 공 발사 방향 조절
  - 풍선 터뜨리기
  - 풍선 분할 메커니즘 이해
- **특징**:
  - 풍선 배열이 단순함
  - 공의 반사 각도 학습
  - 게임 성공 경험 제공

### Target Audience

- 모든 연령대의 캐주얼 게임 플레이어
- 반응속도와 전략을 즐기는 사용자
```

Product Requirements Document (PRD)

팡 (PANG) 게임

Overview

팡 (PANG) 은 고전적인 버블 팝 (Bubble Pop) 게임으로, 플레이어가 발사한 공으로 화면에 떠있는 풍선을 터뜨려 모든 풍선을 제거하는 것이 목표입니다.

게임 컨셉

- **핵심 메커니즘**: 하단에서 공을 발사하여 풍선을 터뜨림
- **풍선 분할**: 큰 풍선은 터질 때 작은 풍선 3 개로 분할됨
- **목표**: 제한된 시도 내에 모든 풍선을 제거
- **장애물**: 회전하는 막대 (바리케이드) 를 피해야 함

Mission 1 특징

초보자용 튜토리얼 레벨

- **풍선 수**: 5 개 (기본 풍선)
- **난이도**: 매우 낮음
- **막대**: 없음 또는 매우 느린 회전 속도
- **목표**: 게임 기본 조작법 학습
 - 마우스/터치로 공 발사 방향 조절
 - 풍선 터뜨리기
 - 풍선 분할 메커니즘 이해
- **특징**:
 - 풍선 배열이 단순함
 - 공의 반사 각도 학습
 - 게임 성공 경험 제공

Coding Agent는 랜덤성에 의해 동작하므로, 실습 결과가 이미지와 다를 수 있습니다.
또한, 위 실습 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

PRD.md 검토

- 원하는 방향과 다른 부분
- AI가 몰라도 되는 정보
- 기술적 정보

Overview

팡 (PANG) 은 고전적인 버블 팝 (Bubble Pop) 게임으로, 플레이어가 발사한 공으로 화면에 떠있는 풍선을 터뜨려 모든 풍선을 제거하는 것이 목표입니다.

게임 컨셉

- 핵심 메커니즘: 하단에서 공을 발사하여 풍선을 터뜨림
- 풍선 분할: 큰 풍선은 터질 때 작은 풍선 3개로 분할됨
- 목표: 제한된 시도 내에 모든 풍선을 제거
- 장애물: 회전하는 막대 (바리케이드) 를 피해야 함

Mission 1 특징

초보자용 튜토리얼 레벨

- 풍선 수: 5 개 (기본 풍선)
- 난이도: 매우 낮음
- 막대: 없음 또는 매우 느린 회전 속도
- 목표: 게임 기본 조작법 학습
 - 마우스/터치로 공 발사 방향 조절
 - 풍선 터뜨리기
 - 풍선 분할 메커니즘 이해
- 특징:
 - 풍선 배열이 단순함
 - 공의 반사 각도 학습
 - 게임 성공 경험 제공

Target Audience

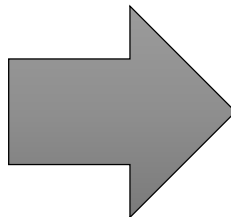
- 모든 연령대의 캐주얼 게임 플레이어
- 반응속도와 전략을 즐기는 사용자

Tech Stack

- Language: TypeScript
- Framework: React 19
- Build Tool: Vite 8
- Code Quality: ESLint with React hooks and refresh plugins
- Game Logic: Custom physics engine for ball collision and reflection

Game Features

- 게임 플레이: 직관적인 터치/클릭 컨트롤
- 풍선 시스템: 다양한 크기와 색상의 풍선
- 물리 엔진: 공의 반사 및 충돌 계산
- 레벨 시스템: 점진적인 난이도 증가 (Mission 1-N)
- 스코어 시스템: 시도 횟수 기반 점수 계산
- 반응형 디자인: 데스크톱 및 모바일 지원



Overview

팡 (PANG) 은 고전적인 버블 팝 (Bubble Pop) 게임으로, 플레이어가 발사한 와이어로 화면에 떠있는 풍선을 터뜨려 모든 풍선을 제거하는 것이 목표입니다.

게임 컨셉

- 핵심 메커니즘: 하단에서 와이어를 발사하여 풍선을 터뜨림
- 풍선 분할: 큰 풍선은 터질 때 작은 풍선 2개로 분할됨
- 목표: 제한된 시도 내에 모든 풍선을 제거
- 장애물: 막대 (바리케이드) 를 피해야 함

Mission 1 특징

초보자용 튜토리얼 레벨

- 풍선 수: 5 개 (기본 풍선)
- 난이도: 매우 낮음
- 막대: 없음
- 목표: 게임 기본 조작법 학습
 - 풍선 터뜨리기
 - 풍선 분할 메커니즘 이해
- 특징:
 - 풍선 배열이 단순함
 - 공의 반사 각도 학습
 - 게임 성공 경험 제공

Target Audience

- 모든 연령대의 캐주얼 게임 플레이어
- 반응속도와 전략을 즐기는 사용자

Game Features

- 게임 플레이: 좌우 방향키, 스페이스바
- 풍선 시스템: 다양한 크기와 색상의 풍선
- 물리 엔진: 풍선의 반사 및 충돌 계산
- 레벨 시스템: 점진적인 난이도 증가 (Mission 1-3)
- 스코어 시스템: 시도 횟수 기반 점수 계산
- 반응형 디자인: 데스크톱 및 모바일 지원

Coding Agent는 랜덤성에 의해 동작하므로, 실습 결과가 이미지와 다를 수 있습니다. 또한, 위 실습 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

PRD.md 주의 사항

Context Window (용어 암기)

- AI가 응답을 생성할 때 참조하는 모든 텍스트, 모델의 “작업 메모리”를 의미
- AI가 참조하는 문서에 불필요한 설명, 중복 코드 등이 많아지면 노이즈 발생
 - AI가 집중할 부분이 약해짐
 - 노이즈가 많으면 답변 정확도가 떨어짐

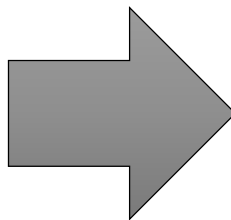
~~PRD.md 에서 개발에 불필요한 내용들을 모두 삭제해~~

AI에게 지시했던 내용을 AI에게 감독을 맡기면 잘 수행하지 못하는 경우가 대부분이다.
따라서 AI에게 지시 후 감독업무는 반드시 **사람**이 한다.

이렇게 AI가 혼자서 결정하지 않고,
중간에 사람이 개입하는 것을 **Human-in-the-loop** 라고 한다.

세부 요구사항 문서를 작성 및 검토

아래 파일을 생성하고 초안을 작성해
docs/FEATURES/main.md : 첫 메인화면 구성
docs/FEATURES/game_rule.md : 게임 룰에 대한 상세 내용
docs/FEATURES/mission1.md : 미션 1에 대한 게임 난이도 및 규칙
이후 이를 @file: AGENTS.md 에 기록해



완료했습니다.

생성된 문서:

- docs/FEATURES/main.md - 메인 화면 구성 (타이틀, 메뉴 버튼, 배경, 컴포넌트 구조)
- docs/FEATURES/game_rule.md - 게임 룰 상세 (발사 메커니즘, 점수 시스템, 승리/패배 조건)
- docs/FEATURES/mission1.md - 미션 1 상세 (난이도, 풍선/장애물 설정, 튜토리얼 가이드)

AGENTS.md 업데이트:

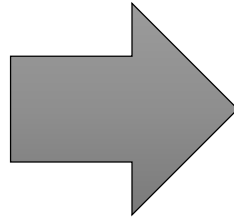
- 문서 목록에 3 개의 새 파일 추가

✓ Created by generative AI in 2m 1s

요구사항 문서 작성 Commit

PRD문서에 대한 작업 및 감독이 완료됨에 따라 Commit 이력 기록

이전 commit으로부터 지금까지 한 작업 내용을 기반으로 commit 작성



```
code56@SDS-DEDU-50006 MINGW64 /c/reviewer/workspace/Pang (master)
$ git log
commit 8ed7ab7a86042b6868c03547b0ce42011aa82702 (HEAD -> master)
Author: unknown <code56@smartdesk.sds.cloud.com>
Date: Tue Apr 28 15:31:10 2026 +0900

    Add project documentation and update AGENTS.md

    - Add docs/PRD.md with complete product requirements

    - Add docs/FEATURES/ with detailed feature specifications

    - Update AGENTS.md to reflect Pang as bubble pop game project

    - Document main features: gameplay, physics, levels, obstacles, scoring
```

Phase 설계

AI는 큰 작업을 할 수록 결과물의 품질이 떨어진다.

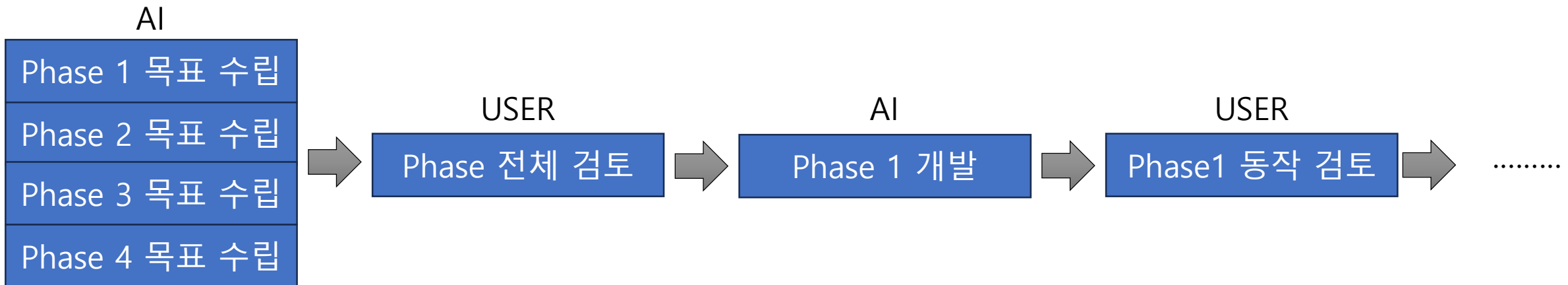
품질을 높이기 위하여 동작되는 SW가 유지되면서, **작은 기능의 단위로 개발**되도록 구성한다.

프로그래밍 개발 경험이 많을수록 아래의 판단을 더욱 잘 할 수 있다.

- 어떤 Phase들이 어떤 순서로 필요할 것인가?
- 어느정도 크기의 기능이 AI의 답변 품질을 유지할 수 있는 정도인가?

이후 작업에선 프로그래밍 개발 경험이 적은 개발자라 가정하고

Phase단계를 수립하고 개발할 것이다.



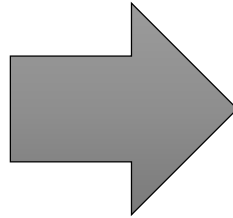
PLAN.md 준비

Phase별 설명 + TODO 의 내용을 기록한 문서

개발이 완료된 후 버리는 문서

- 참고) TODO.md : TODO 내용만 존재(자세한 설명이 없음)

docs/PLAN.md의 빈 파일을 하나 만들고
@file: AGENTS.md 에 해당 문서는 Phase별 목표를 세운 파일이라고 기록



Development Guidelines

This project is developed based on the following documents.

- docs/PRD.md : Complete requirements definition
- docs/PLAN.md : Phase goals and milestones
- docs/FEATURES/* : Feature-specific detailed requirements

Main Features:

- Bubble pop gameplay (wire firing, balloon popping, balloon splitting)
- Physics engine (balloon reflection and collision calculations)
- Level system (progressive difficulty increase from Mission 1)
- Bar obstacles (barricades)
- Score system (based on attempt count)

Target Audience: Casual game players of all ages

Project Structure

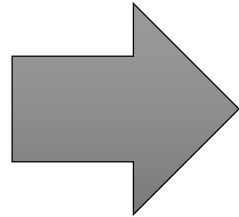
```
Pang/
├── src/
│   ├── assets/           # Static assets (images, fonts, sounds)
│   ├── components/       # Game components (Ball, Balloon, Bar, etc.)
│   ├── hooks/            # Custom React hooks
│   ├── utils/            # Game logic utilities (physics, collision)
│   ├── App.tsx           # Root application component
│   ├── main.tsx          # Application entry point
│   ├── App.css           # Root component styles
│   └── index.css         # Global styles
├── docs/
│   ├── PRD.md            # Product Requirements Document
│   └── FEATURES/         # Feature-specific requirements
├── public/               # Public static files
├── package.json          # Dependencies and scripts
├── tsconfig.json         # TypeScript configuration
├── vite.config.ts        # Vite configuration
└── eslint.config.js      # ESLint configuration
```

Coding Agent는 랜덤성에 의해 동작하므로, 실습 결과가 이미지와 다를 수 있습니다.
또한, 위 실습 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

PLAN.md 초안 작성

AI에게 PLAN.md에 대한 초안 작성을 지시

@file: PLAN.md 에 Phase별 목표 초안을 만들어
각 Phase별로 동작되는 SW가 되어야 함
사람인 내가 직접 잘 동작되는지 테스트해보고 피드백을 줄 예정
날 고객님이라고 생각하고,
Phase 별로 어떤 기능을 목표로 만들었고,
고객님은 어떤 부분을 중심으로 보면 될지 잘 설명해



완료했습니다.

📄 Phase 별 요약 (고객님 관점)

Phase	목표	고객님 테스트 포인트
Phase 1	기본 게임 루프	와이어 발사, 풍선 터뜨리기, 분할 메커니즘
Phase 2	현실적인 물리	반사 각도, 연속 터뜨리기, 부드러운 애니메이션
Phase 3	난이도 & 전략	3 단계 미션, 막대 장애물 회피, 점진적 어려움
Phase 4	경쟁 요소	스코어 기록, 스타 등급, 재도전 동기

각 Phase 는 독립적으로 실행 가능한 완성된 소프트웨어입니다. Phase 1 부터 시작해 각 단계마다 직접 플레이하고 피드백 주시면, 다음 Phase 에 반영하겠습니다.

PLAN.md

팡 (PANG) 개발 계획서

개발 철학

각 Phase 는 독립적으로 실행 가능한 완성된 소프트웨어입니다.
고객님 (사용자) 이 직접 테스트하고 피드백을 주실 수 있도록 설계되었습니다.

Phase 1: 핵심 게임플레이 구현

🎯 목표

와이어를 발사하여 풍선을 터뜨리는 기본 게임 루프 완성

🌟 구현 기능

- 와이어 발사 시스템
 - 스페이스바로 와이어 발사
 - 좌우 방향으로 발사 각도 조절
 - 와이어의 직선 이동 애니메이션
- 풍선 시스템
 - 화면에 풍선 5 개 표시 (Mission 1 기본)
 - 와이어 충돌 시 풍선 터지는 효과
 - 풍선 분할: 큰 풍선 → 작은 풍선 2 개로 분할
- 게임 상태 관리
 - 시작 화면 → 게임 중 → 성공/실패 화면 전환
 - 모든 풍선 제거 시 성공
 - 시도 횟수 제한 시 실패

👤 고객님 테스트 포인트

1. 조작 직관성: 방향키와 스페이스바로 와이어를 쉽게 조절할 수 있는지
2. 충돌 감지: 와이어가 풍선을 정확히 맞추는지 (누락되거나 오작동 없는지)
3. 풍선 분할: 큰 풍선이 터졌을 때 작은 풍선이 자연스럽게 분할되는지
4. 성공/실패 조건: 모든 풍선을 터뜨리면 성공, 실패 조건이 명확한지

Phase 2: 물리 엔진 고도화

🎯 목표

현실적인 반사 및 충돌 물리 구현으로 게임 몰입도 향상

🌟 구현 기능

- 정교한 반사 시스템
 - 풍선 표면에서의 와이어 반사 각도 계산 (입사각 = 반사각)
 - 연속 반사 지원 (한 번 발사로 여러 풍선 터뜨리기)
 - 벽면 반사 처리
- 충돌 최적화
 - 원형 충돌 감지 정확도 향상
 - 와이어 경로 최적화 (불필요한 계산 제거)
 - 성능 개선 (60fps 유지)
- 애니메이션 개선
 - 와이어 이동 부드러움
 - 풍선 터질 때 파티클 효과
 - 분할 풍선 애니메이션

👤 고객님 테스트 포인트

1. 반사 각도: 와이어가 풍선에 닿았을 때 반사 각도가 자연스러운지
2. 연속 터뜨리기: 한 번 발사로 여러 풍선을 연속으로 터뜨릴 수 있는지
3. 성능: 와이어가 빠르게 움직여도 프레임 드롭 없이 부드러운지
4. 시각적 효과: 풍선이 터질 때 애니메이션이 만족스러운지

Phase 3: 레벨 시스템 & 장애물

🎯 목표

점진적인 난이도 증가와 전략적 요소 추가

🌟 구현 기능

- 다중 레벨 시스템
 - Mission 1: 초보자 (풍선 5 개, 막대 없음)
 - Mission 2: 중급 (풍선 8 개, 막대 2 개)
 - Mission 3: 고급 (풍선 10 개, 막대 4 개)
 - 레벨 완료 시 다음 레벨 자동 진행
- 막대 (Bar) 장애물
 - 와이어가 막대에 닿으면 발사 종료
 - 다양한 위치와 각도에 막대 배치
 - 전략적 회피 필요
- 레벨 선택 화면
 - 완료한 레벨 표시
 - 재도전 기능
 - 레벨별 난이도 표시

👤 고객님 테스트 포인트

1. 난이도 곡선: Mission 1 → 2 → 3 으로 자연스럽게 어려워지는지
2. 막대 전략: 막대를 피하기 위해 발사 각도를 조절해야 하는 전략적 요소가 있는지
3. 레벨 진행: 레벨을 클리어했을 때 성취감이 있는지
4. 재도전: 실패했을 때 다시 도전하고 싶은 동기가 있는지

Coding Agent는 랜덤성에 의해 동작하므로, 실습 결과가 이미지와 다를 수 있습니다.
또한, 위 실습 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

Phase 4: 스코어 & 완성도

🎯 목표

경쟁 요소와 재방문 동기 추가

🌟 구현 기능

- 스코어 시스템
 - 시도 횟수 기반 점수 계산 (적은 시도 = 높은 점수)
 - 레벨별 최고 점수 기록
 - 스코어보드 표시
- 스타 등급 시스템
 - 시도 횟수에 따라 1~3 스타 부여
 - 레벨별 총 스타 수 표시
- 게임 완료 화면
 - 전체 미션 완료 시 축하 화면
 - 총 점수와 총 스타 표시
 - 재플레이 유도
- 소음 효과 (선택)
 - 와이어 발사 소리
 - 풍선 터지는 소리
 - 성공/실패 효과음

👤 고객님 테스트 포인트

1. 점수 동기: 적은 시도로 클리어하려는 동기가 생기는지
2. 최고 점수: 자신의 기록을 깨고 싶은 욕구가 있는지
3. 스타 시스템: 3 스타를 받기 위해 노력하고 싶은지
4. 재방문: 게임을 다시 하고 싶은지, 친구에게 추천하고 싶은지

📅 Phase 별 테스트 체크리스트

Phase	핵심 테스트 항목	예상 소요 시간
Phase 1	조작, 충돌, 분할, 성공/실패	5-10 분
Phase 2	반사 각도, 연속 터뜨리기, 성능	5-10 분
Phase 3	난이도 곡선, 막대 전략, 레벨 진행	10-15 분
Phase 4	스코어 동기, 재방문 욕구	5-10 분

🗣️ 피드백 수집 방법

각 Phase 완료 후 다음 사항을 피드백해 주세요:

1. 작동 여부: 모든 기능이 정상 작동하는지
2. 사용자 경험: 조작이 직관적이고 즐거운지
3. 버그/이슈: 발견한 문제점 (충돌 오작동, 화면 깨짐 등)
4. 개선 제안: 더 재미있게 만들기 위한 아이디어

🚀 개발 일정 (예상)

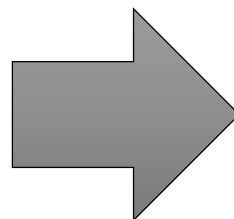
- Phase 1: 2-3 일 (핵심 게임플레이)
- Phase 2: 2-3 일 (물리 엔진)
- Phase 3: 3-4 일 (레벨 시스템)
- Phase 4: 2-3 일 (스코어 & 완성도)

총 예상 기간: 9-13 일 (Phase 별 피드백 반영 시간 포함)

PLAN.md

현재 단계는 초안으로 원하는 정도의 설계가 아닐 가능성이 높다.
AI와 대화를 통하여 PLAN.md의 내용을 수정한다.

지금의 Phase보다 좀 더 많은 Phase구성으로 작게 나눠
Phase1은 먼저 메인화면만 뜨고, 게임 선택 / 시작버튼 / 게임종료
이런 내용들이 들어가야함
Phase별로 개발이 되면 사람이 직접 실행하고, 해당 기능을 테스트할 수 있어야함
개발 일정에 대한 소요시간들은 전부 필요 없음



완료했습니다. Phase 를 16 단계로 세분화하여 각 단계마다 독립적으로 테스트 가능하도록 재구성했습니다.

📄 주요 변경 사항

항목	이전	변경 후
Phase 수	4 개	16 개
Phase 1	핵심 게임플레이	메인 메뉴 UI 만
진행 방식	큰 기능 단위	한 번에 하나의 기능
개발 일정	포함	제거

🎮 Phase 1-7 요약 (기초 완성)

- Phase 1: 메인 메뉴 (시작/종료 버튼)
- Phase 2: 화면 전환 (메인 ↔ 게임)
- Phase 3: 발사체 조작 (방향키 + 스페이스)
- Phase 4: 풍선 5 개 표시
- Phase 5: 충돌 감지
- Phase 6: 풍선 분할
- Phase 7: 성공/실패 조건

🚀 Phase 8-16 요약 (고도화)

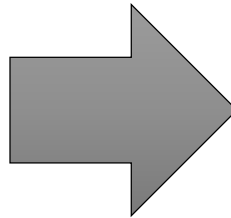
- Phase 8-9: 물리 엔진 + 장애물
- Phase 10: 3 단계 미션 시스템
- Phase 11-12: 스코어 + 스타 등급
- Phase 13-15: 애니메이션, 사운드, 반응형
- Phase 16: 최종 다듬기

Coding Agent는 랜덤성에 의해 동작하므로, 실습 결과가 이미지와 다를 수 있습니다.
또한, 위 실습 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

Design

Phase별 구현을 하기 전에 어떻게 만들 것인지 Design(설계문서)를 받고 검토가 필요
전체 Design을 하나로 받는 것이 아닌 Phase별 설계문서가 필요하다

docs/design/phase1.md를 만들고,
phase1을 어떻게 만들 것인지 설계내용을 작성해봐
내가 검토할꺼야



docs/design/phase1.md 파일을 생성했습니다.

설계 문서 내용:

1. 아키텍처 설계: 컴포넌트 구조 (MainMenu → Header, MenuButtons → StartButton, ExitButton)
2. 파일 구조: 각 컴포넌트별 TSX 와 CSS 파일 분리
3. 컴포넌트 상세: 각 컴포넌트의 인터페이스, Props, 구현 예시
4. 디자인 가이드: 색상 팔레트, 타이포그래피, 애니메이션
5. 인터랙션 설계: 호버/클릭 효과
6. 테스트 체크리스트: 기능, UI/UX, 반응형 테스트 항목
7. 구현 단계: 6 단계별 상세 작업 내용
8. 예상 이슈 및 대응: window.close() 제한, 폰트 로딩, 모바일 깨짐
9. 예상 개발 시간: 총 4 시간

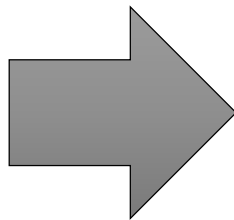
검토 부탁드립니다. 수정이 필요한 부분이 있다면 알려주세요.

/ Created by generative AI in 1m 35s

Design 문서 연결

Design 문서 CLAUDE.md에 연결

이 내용도 CLAUDE.md에 기록해

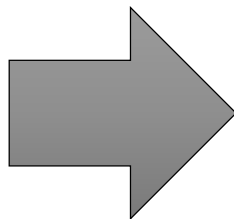


CALUDE.md에 다음 내용을 추가했습니다:

1. **Development Guidelines** 섹션: docs/design/phase1.md 문서 참조 추가
2. **Project Structure** 섹션: docs/design/phase1.md 파일 구조 추가

✓ Created by generative AI in 1m 39s

지금 CALUDE.md에 기록한 방식은 phase1 design문서만 직접적으로 연결시켰는데,
추후 여러 phase가 추가될 예정이기에 그에 맞게 사전에 기록 방식을 수정해놔



CALUDE.md를 확장 가능한 방식으로 수정했습니다:

1. **Development Guidelines**: docs/design/phase*.md 로 변경 (모든 phase 문서 참조)
2. **Project Structure**: phase1.md, phase2.md, ... 로 패턴화 (추가 phase 자동 포함)*

✓ Created by generative AI in 1m 4s

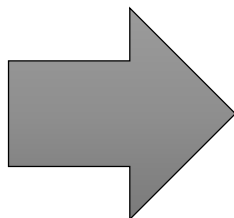
Implement

앞에서 생성한 design을 기반으로 실제 구현 시작

PLAN.md에서 phase1 내용만 개발을 해줘

- 이 파일 설계안은 `@file: phase1.md` 를 참고해

개발이 끝나면 내가 직접 phase1을 검토할 것이며,
phase1 내용 외에 추가로 개발해선 안돼



구현 완료 내용

메인 메뉴 화면이 설계 문서에 따라 구현되었습니다:

생성된 파일

```
src/components/MainMenu/  
├── Header.tsx / Header.css      - PANG 타이틀 로고  
├── StartButton.tsx / StartButton.css - 게임 시작 버튼 (초록색)  
├── ExitButton.tsx / ExitButton.css  - 게임 종료 버튼 (빨간색)  
├── MenuButtons.tsx / MenuButtons.css - 버튼 컨테이너  
└── MainMenu.tsx / MainMenu.css     - 메인 메뉴 컨테이너
```

기능

- PANG 타이틀: 주황색 굵은 폰트로 표시 (#FF9800)
- 게임 시작 버튼: 초록색 (#4CAF50), 호버/클릭 효과 적용
- 게임 종료 버튼: 빨간색 (#f44336), `window.close()` 호출
- 배경: 파란색 그라데이션 (#64B5F6 → #42A5F5)
- 버튼 인터랙션: 호버 시 색상 변경, 클릭 시 축소 효과 (scale 0.95)

검증 결과

- ☒ TypeScript 컴파일 성공
- ☒ Vite 빌드 성공 (191KB bundle)

`npm run dev` 로 개발 서버를 실행하면 메인 메뉴 화면을 확인할 수 있습니다.

✓ Created by generative AI in 7m 6s

Coding Agent는 랜덤성에 의해 동작하므로, 실습 결과가 이미지와 다를 수 있습니다.
또한, 위 실습 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

Implement

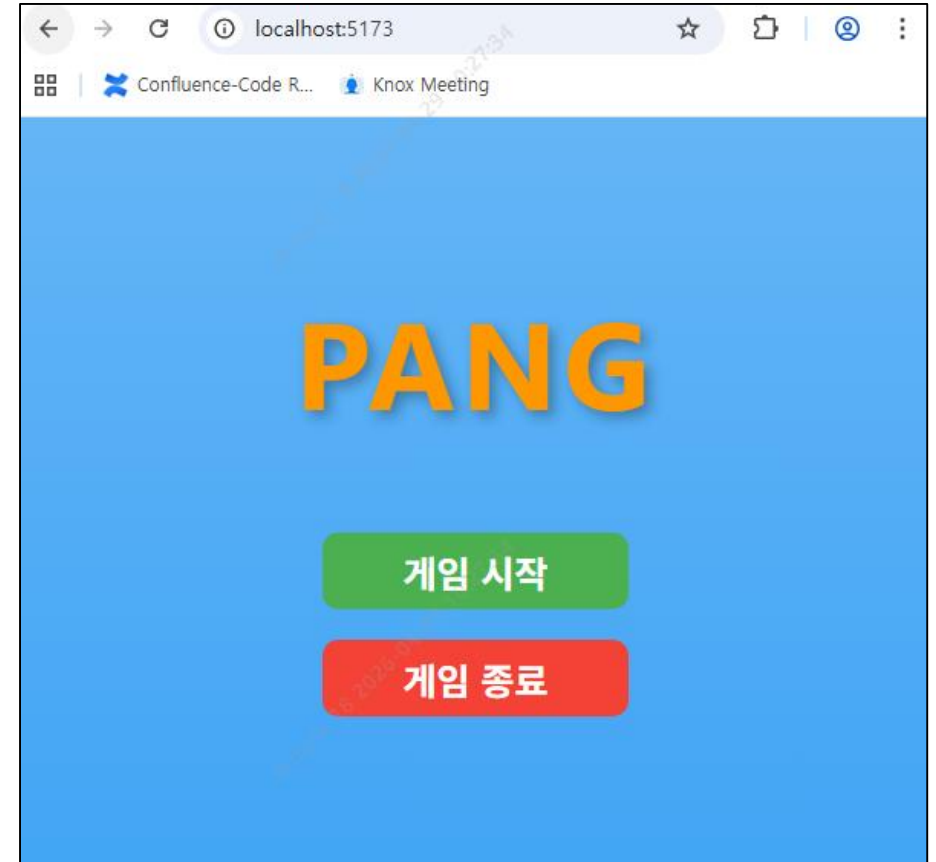
직접 실행하여 phase1에 대한 개발이 정상적으로 되었는지 검사

```
code56@SDS-DEDU-50006 MINGW64 /c/reviewer/workspace/Pang (master)
$ npm run dev

> vite-project@0.0.0 dev
> vite

VITE v8.0.10 ready in 346 ms

→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```



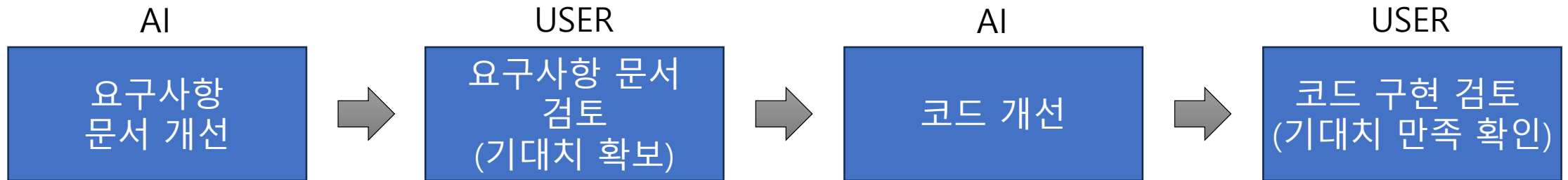
Coding Agent는 랜덤성에 의해 동작하므로, 실습 결과가 이미지와 다를 수 있습니다.
또한, 위 실습 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

Improvement

키보드로도 메뉴를 움직이도록 개선하고 싶은 경우
아래와 같이 바로 수정하도록 요청하면 안된다.

~~"키보드 방향키로 메뉴를 이동할 수 있게 코드를 수정해줘"~~

항상 아래와 같은 흐름으로 개발, 수정 절차를 지키자



AI가 어떻게 개발할지 미리 감독관이 알고 있는 상태에서 (기대치 확보)

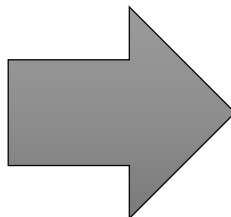
AI가 작성을 하고, 그 작성된 코드가 감독관과 합의된 내용으로 작성됐는지 리뷰해야 한다.

- 본래는 코드를 직접 리뷰할 필요가 있지만, 이번 실습에선 코드 리뷰는 생략한다.

Improvement

요구사항 문서 개선

키보드 방향키로 메뉴 이동을 할 수 있게 먼저 `@file: main.md` 를 개선해봐



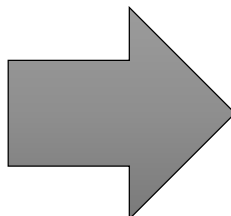
docs/FEATURES/main.md 파일을 개선했습니다.

추가된 내용

3. 키보드 네비게이션 섹션을 새로 추가:

- 방향키 상하 (↑/↓): 메뉴 항목 간 이동 (loop 기능)
- Enter/Space 키: 선택된 메뉴 항목 실행
- 초기 포커스: 첫 번째 메뉴 항목에 자동 포커스
- 시각적 피드백: 포커스된 항목 하이라이트 표시

해당 부분에 대한 것을 `@file: phase1.md` 도 반영하여 개선



Diff: \reviewer\workspace\Pang\docs\design\phase1.md

```
11 - 게임 타이틀 "PANG" 로고 표시
12 - 메인 메뉴 버튼 (게임 시작, 게임 종료) 구현
13 - 버튼 인터랙션 및 시각적 피드백 제공
14 - **키보드 네비게이션 지원 (방향키, Enter)**
```

```
32
33 Phase 1에서는 전역 상태 관리가 필요하지 않습니다. 각 컴포넌트 내부에서 로컬 상태만 사용합니다.
34
```

```
35 **키보드 네비게이션 상태**:
36 - `focusedIndex`: 현재 포커스된 메뉴 항목 인덱스 (0: 게임 시작, 1: 게임 종료)
```

Show All (604 lines)

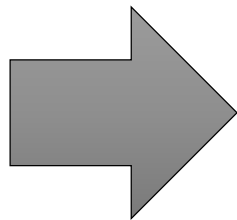
docs/design/phase1.md 파일을 키보드 네비게이션 기능을 포함하도록 개선했습니다.

Coding Agent는 랜덤성에 의해 동작하므로, 실습 결과가 이미지와 다를 수 있습니다.
또한, 위 실습 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

Improvement

코드 개발

설계 내용을 반영하여 키보드 방향으로 메뉴를 움직일 수 있도록 개발
이후 내가 직접 실행하여 테스트할 예정
phase1 내용 외의 추가개발 절대 불가



Phase 1 키보드 네비게이션 기능 구현이 완료되었습니다.

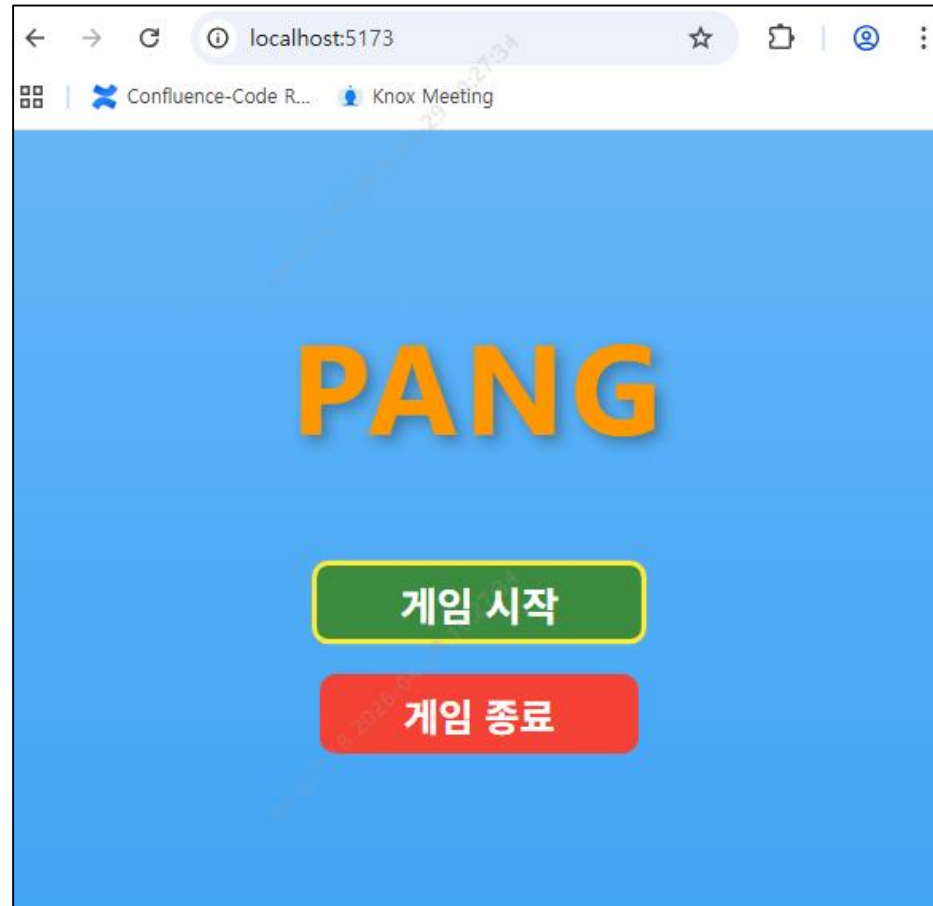
구현 완료 내용

변경된 파일

- **MainMenu.tsx:** `focusedIndex` 상태, 키보드 이벤트 핸들러 (`handleKeyDown`), `useEffect` 로 이벤트 리스너 등록
- **MenuButtons.tsx:** `focusedIndex` props 추가, 각 버튼에 `isFocused` 전달
- **StartButton.tsx:** `isFocused` props, `tabIndex`, `aria-current` 추가
- **ExitButton.tsx:** `isFocused` props, `tabIndex`, `aria-current` 추가
- **StartButton.css:** `.focused` 클래스 (진한 초록색 배경, 노란색 테두리, scale 1.05)
- **ExitButton.css:** `.focused` 클래스 (진한 빨간색 배경, 노란색 테두리, scale 1.05)

Improvement

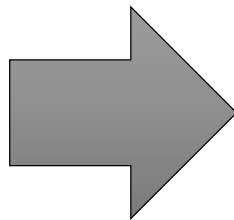
수정된 요구사항이 반영되었는지 직접 실행하여 확인



phase1 개발 완료 commit

기능 개발에 대한 검토까지 완료되었으니 commit 이력 기록

phase 개발과 관련된 내용을 commit



```
code56@SDS-DEDU-50006 MINGW64 /c/reviewer/workspace/Pang (master)
$ git log
commit 18c6f03dd0071763185a3121d6d8febee3454a57 (HEAD -> master)
Author: unknown <code56@smartdesk.sds.cloud.com>
Date:   Wed Apr 29 13:11:48 2026 +0900

    feat: Implement Phase 1 main menu with keyboard navigation

    - Add MainMenu component structure (Header, MenuButtons, StartButton, ExitButton)

    - Implement keyboard navigation (ArrowUp/Down, Enter, Space)
:...skipping...
```

[도전] 모든 Phase 완성하기

phase1을 개발한 것과 마찬가지로 나머지 phase에 대해서도 마저 개발하면 높은 품질의 작품을 만들 수 있다.

phase별 개발의 장점

1. 버그를 사전에 찾아내고 최소한의 범위로 수정을 진행할 수 있다.
 - AI가 작업하기 전에 사람이 생각하는 바와 Sync를 맞춰 개발 전에 버그 발생을 방지 가능
 - Phase 범위로 코드가 수정되기에 해당 범위 정도만의 코드에 대한 분석만으로 버그 수정 가능
2. 요구사항 분석, 정의가 개발 중간에도 추가, 개선될 수 있음
3. 생각치 못한 내용에 대하여 유연하게 대처 가능

Chapter 04

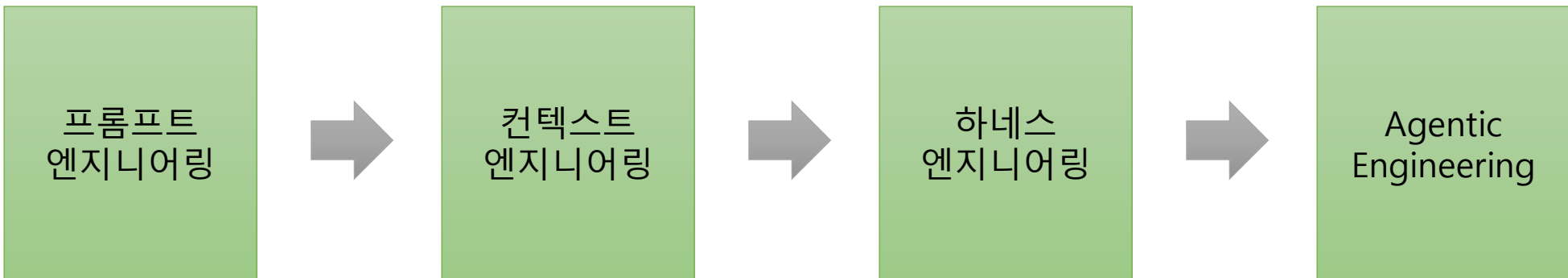
Coding Agent의 Engineering 발전 과정

SW 품질을 위한 Agentic Engineering

Coding Agent의 Engineering 발전 과정

이번 챕터에서는 Coding Agent의 활용과 함께 발전해 온 Engineering의 흐름을 살펴본다.

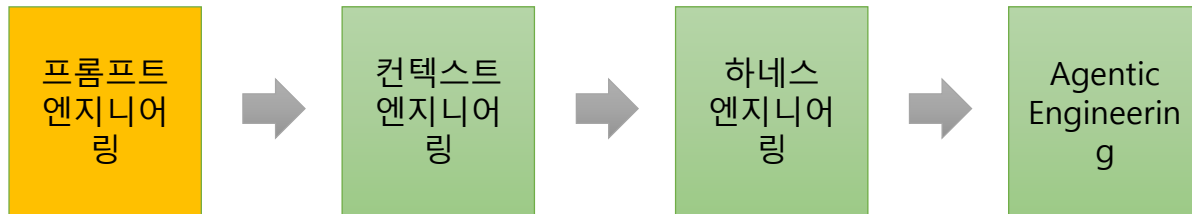
이 과정을 통해 각 엔지니어링 방법론에서 무엇이 중요했는지 이해하고,
나아가 Agentic Engineering에서는 무엇에 집중해야 하는지 파악할 수 있다.



Prompt Engineering (2020년)

2020년 당시 User -> LLM -> Answer 와 같이 입력을 보고 한번에 답을 생성하는 단순한 구조였으며, 당시에는 프롬프트 엔지니어링이 중요했음.

이 시기에는 결과의 품질이 대부분 프롬프트(입력 문장)에 의해 결정되었기 때문에, 어떻게 질문하고 어떤 맥락을 제공 하느냐에 초점을 맞추고, TIP들이 모여 Engineering이라는 이름으로 정리되었음



Prompt Engineering 예시

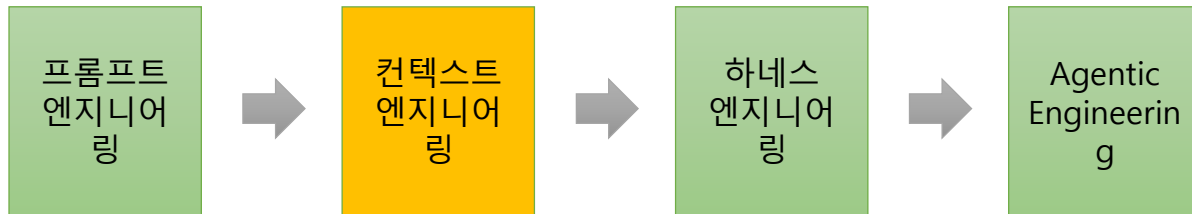
- Few-shot : 예제 몇 개 알려주면서 질문
- CoT : 단계별로 생각해줘 라는 키워드와 함께 질문
- Role 프롬프팅 : 역할 부여, 페르소나 라고 불리었음
- Output Format 지정 : 결과 형식을 요청
- Reflection : 답이 맞는지 한번 더 검토하라고 지시

Context Engineering (2023년)

LLM은 한 번에 처리할 수 있는 입력의 범위인 Context Window를 가진다.

이 제한된 범위 안에 어떤 정보를, 어떻게 구조적으로 넣을 것인가를 다루는 접근이 바로 Context Engineering이다.

핵심은 프롬프트에 포함될 정보 (Context Window)를 구조적으로 설계하는 방법이다.

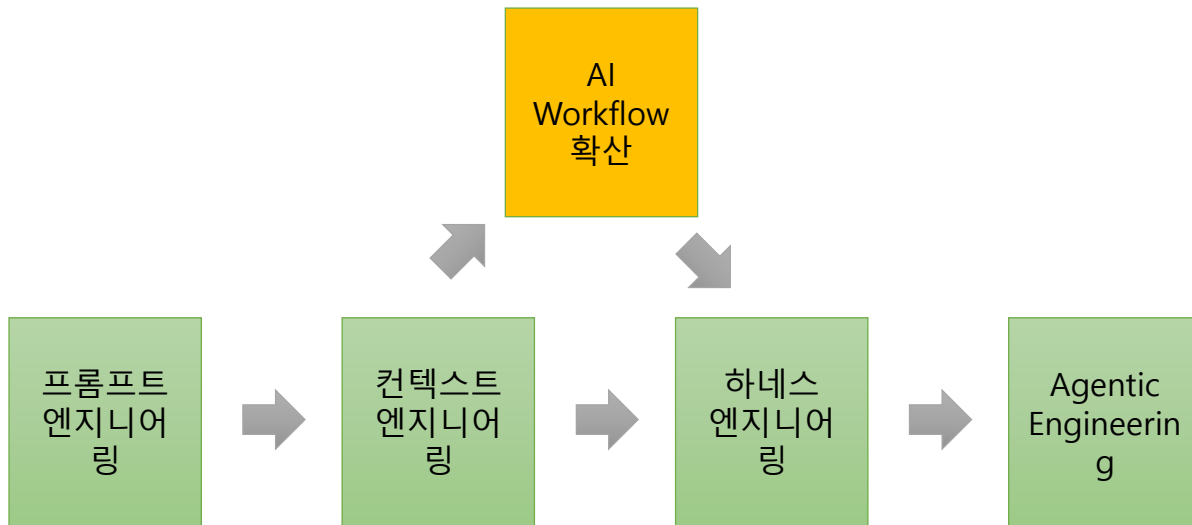


Context Engineering 예시

- 시스템 프롬프트 설계 (Rule 파일 등으로 구성)
- RAG, 메모리, 멀티턴 히스토리 구성
- 필요한 정보만 선별하여 효율적으로 전달하는 전략

AI Workflow 확산 (2023년)

한 번의 질문에 한 번의 답을 생성하는 1회성 LLM 동작만으로는 실제 AI 서비스를 구현하기 어려웠다. **단일 LLM 호출이 아니라, 여러 단계에 걸쳐 LLM을 반복적으로 활용**하여 최종 결과를 만들어낸다. 결국, 실제 AI 서비스는 단일 동작이 아닌 Step 단위로 구성된 Workflow가 필요하다는 것이 명확해졌고, 이러한 흐름을 설계하고 관리하기 위한 도구로 LangGraph와 같은 Workflow 기반 라이브러리들이 주목받기 시작했다.



Agent로 구성되는 Workflow 패턴 양상

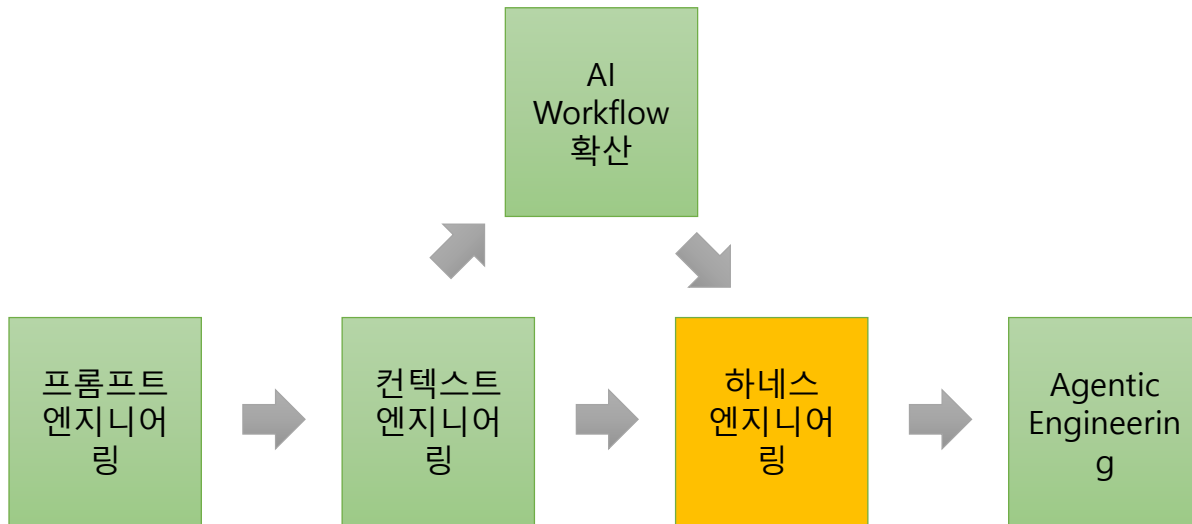
- 순차적(Sequential) Workflow
- 병렬적(Parallel) Workflow
- 평가-개선(Evaluator-optimizer) workflow

Harness Engineering (2025년)

AI Agent에서 “하네스(Harness)”는 Agent가 장시간으로 실행되더라도 올바른 임무를 수행하도록 하는 방법론이다.

에이전트 자체의 성능을 높이는 것보다, Workflow 레벨에서 안전장치를 설계하는 것을 하네스 엔지니어링이라 한다.

여기서 중요한 키워드는 **장시간 실행**이다.



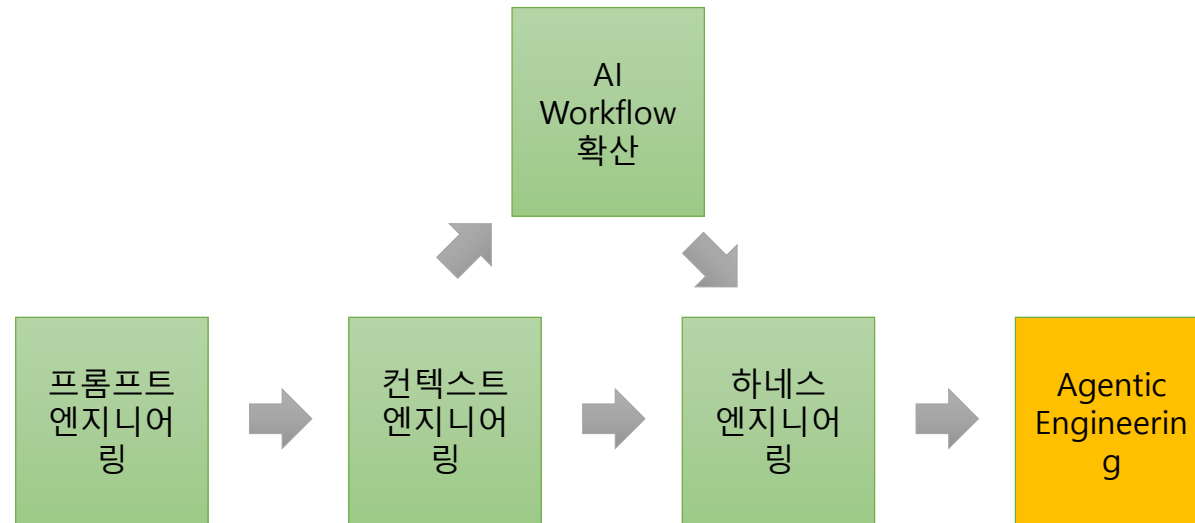
하네스 전 / 후

Agentic Engineering (2026)

Coding Agent로 인해 생산성은 높아지고, 기능도 잘 동작하지만,

SW 품질 저하로 고객에게 납품하기 힘든 제품이 만들어지는 문제 발생

1. 코드에 대한 이해에 대한 필요성이 적어지기 때문에 의도에 맞지 않는 숨어 있는 버그, 오버엔지니어링이 발생
2. 전체 코드 구조에 대한 이해도가 떨어져, 성능 최적화 포인트나 개선 아이디어를 도출하기 어려워 짐
3. 결과적으로 제품 SW 품질에 대한 신뢰와 자신감이 없음



Chapter 05

비용 관점으로 바라본 Agentic Engineering

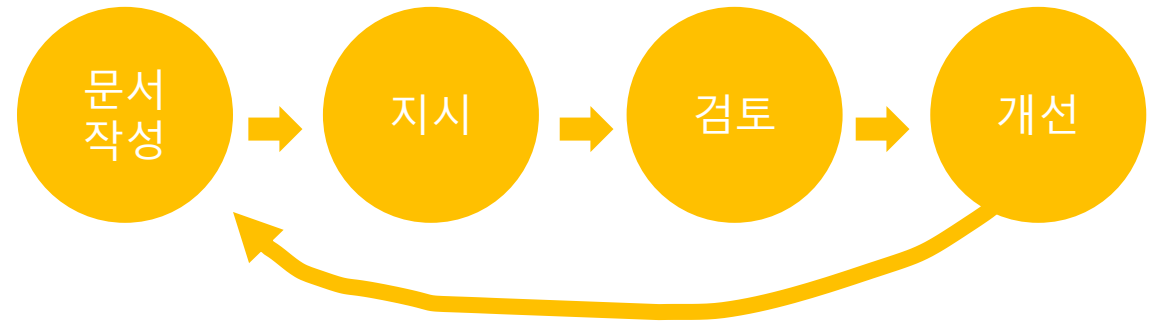
SW 품질을 위한 Agentic Engineering

ROI 관점에서 본 Agentic Engineering - 1

Agentic Engineering의 행동은 다음과 같다.

다음 작업을 위한 문서 작성

문서를 검토하고, 수정, 개선한다.
AI와 토론하며 내용을 보완한다.



지시하기

문서 내용을 기반으로 수행하도록 지시한다.

결과물 확인하기

결과물에 이해가 가지 않는 내용은 AI와 토론
문서가 구체적이지 않아 결과가 잘못 되었다면 문서부터 수정 후, 코드 변경을 지시한다.
마음에 드는 코드가 될 때까지 AI와 협업 하여 수정한다.

ROI 관점에서 본 Agentic Engineering - 2

Agentic Engineering에서의 비용은 크게 세 가지로 나눌 수 있다.

1. 플랜 문서 작성 및 검토 비용 : 코드 수정 비용 대비, 투자 효율이 좋다.
2. 실제 AI가 구현하는 비용 : 낮다.
3. 코드 리뷰 비용 : 매우 크다.

Agentic Engineering에서 ROI를 높이는 방법 - 1

Agentic Engineering에서의 ROI는 결국 리뷰 비용을 얼마나 줄이느냐에 달려 있다.

이를 위해서는 가독성과 분해 능력을 중심으로 설계해야 한다.

1) 가독성 확보 (클린코드)이 좋을수록 리뷰 비용은 줄어든다.

- 가독성이 좋을수록
 - 코드 이해 시간이 줄어들고,
 - 리뷰 속도가 빨라지며,
 - 전체 개발 시간이 단축된다.
- 클린코드는 곧 비용 절감이다.

Agentic Engineering에서 ROI를 높이는 방법 - 2

2) 리뷰 가능한 단위로 분해 (Phase 설계)

- 코드를 한 번에 크게 생성하는 것이 아니라, 사람이 충분히 검토할 수 있는 단위로 나누어 개발해야 한다.
- “리뷰 가능한 크기”로 나누는 것이 핵심이다.

3) 개발 역량 강화

- 개발자의 역량은 Agentic Engineering의 효율을 크게 좌우한다.
- 클린코드에 대한 고민과 리팩토링에 대한 경험이 많을수록, 일관된 가독성을 가진 코드를 설계할 수 있다.
- 개발 경험이 풍부할수록, AI가 생성할 코드의 형태를 예측할 수 있고, 리뷰 가능한 수준으로 Phase를 나누어 계획할 수 있다.
- 자료구조 / 알고리즘 이해도가 높을수록, 문제 분해 능력이 향상되며 AI가 사용할 구현 방식까지 예측할 수 있다.

[도전] Phase별 개발에서 Plan 문서 수정

Phase별 개발에서 작성한 Plan 문서를 바탕으로 (Pang KATA), 사람이 리뷰 가능한 수준으로 Phase를 다시 설계해보자.

단순히 작업 단계를 나누는 것이 아니라,

각 Phase가 명확한 의도와 검증 기준을 갖고, 사람이 이해하고 판단할 수 있도록 구조화 하는 것이 목표다.

수행방법

1. 기존 Plan 문서를 분석한다.
2. 작업을 의미 있는 단위(Phase)로 다시 분리한다. (리뷰 가능한 단위로 나눌 것)
3. 각 Phase에 아래 내용을 포함한다
 - 목적 (Why)
 - 수행 작업 (What)
 - 검증 방법 (Verify)
 - 리뷰 포인트 (Review)

Agentic Engineering

활용 전략

교육 내용 및 학습 목표

- 유지보수 과정에서도 Agentic Engineering을 체계적으로 적용하는 방법을 이해한다.
- AI가 만든 코드를 바라볼 때, '좋은 코드'가 무엇인지 고민해보고, 리팩토링 KATA로 실습을 해 본다.
- 이후 PoC 수집에 대해 알아보고, Coding Agent 기능 중 SubAgent를 Agentic Engineering 관점에서 살펴본다.

Chapter 06

Agentic Engineering을 적용한 유지보수

SW 품질을 위한 Agentic Engineering

공학적인 관점의 전환

유지보수 과정에서도 Agentic Engineering을 체계적으로 적용하는 방법을 이해해보자.

먼저 유지보수는 다음과 같은 목적으로 분류할 수 있다.

1. 기능 추가
2. 버그 수정
3. 버전 업데이트(마이그레이션)
4. 성능 개선

이 중에서도 기능 추가와 버그 수정이 가장 빈번하게 발생한다.

이번 챕터에서는 유지보수의 주요 활동인 기능 추가와 버그 수정을 중심으로, Agentic Engineering을 적용하는 방법을 살펴본다.

Agentic Engineering을 적용한 기능 추가

기능 추가 업무를 AI에게 맡기기 전에는, AI가 요구사항을 정확히 이해했는지 반드시 검증해야 한다.

또한 AI가 수행할 동작이 충분히 예측 가능한 수준까지 사전 논의가 이루어진 이후에, 개발을 지시해야 한다.

Agentic Engineering - 기능 추가 절차

1. Branch 생성 (Sandbox 구성)
2. 요구사항 정리 (AI와 함께 명확화)
3. 설계 문서 작성 (AI 활용 및 인간과 토론)
4. 설계 및 요구사항 검토 (인간 Review)
5. 개발 지시 및 구현 진행
6. 코드 리뷰 수행
7. Merge 수행

Agentic Engineering을 적용한 버그 수정

- 버그 수정은 단순히 코드를 고치는 작업이 아니라, 문제의 원인을 정확히 밝혀내는 과정이다.
- AI에게 바로 수정을 맡기는 방식은 증상만 제거하고, 근본 원인을 놓치는 결과를 초래할 수 있다.
- 이 근본 원인은 AI만 알면 안되고, 인간이 충분히 상황을 이해해야 한다.
- 만약 이해가 가지 않는다면, 수정작업 전, 끝까지 원인을 파헤쳐 알아야 한다.
그래야 예상치 못한 추가 버그가 일어나지 않는다.

Agentic Engineering - 버그수정 절차

1. Branch 생성 (Sandbox 구성)
2. 원인 분석 (AI와 함께 토론으로 명확화)
3. 재현 Scenario 및 재현 Test Case 확보
4. 수정 방향 검토 (AI가 작성, 인간의 검토)
5. 버그 수정 진행
6. 코드 리뷰 수행 & 재현 Test Case Pass 확인
7. Merge 수행

Chapter 07

좋은 코드 판정하기

SW 품질을 위한 Agentic Engineering

좋은 코드는 무엇일까?

- Agentic Engineering은 AI의 결과물을 인간이 검토하고 책임지는 개발 방식이다.
- 이번 챕터에서는 AI가 생성한 코드를 사람이 검토할 때,
어떤 관점으로 코드를 평가해야 하는지를 가볍게 살펴본다.

로버트 C 마틴의 클린코드

- 읽기 쉬운 코드가 좋은 코드이다. 가독성이 최우선이다.
- 이해하기 쉬운 코드는 곧 유지보수가 좋은 코드이다.
- 이름에는 의미가 있어야 하며, 함수는 한가지 역할을 담당해야 한다.
- 코드도 글처럼 읽히도록 해야 하며, 하나의 문서처럼 취급해야 한다.
- 클래스는 하나의 책임을 갖도록 한다. (SRP)
- 테스트코드는 필수이며, 자주 발생하는 중복은 제거해야 한다.

AI시대에 클린코드(가독성이 좋은 코드)는 어떤 장점이 있을까?

AI 코드리뷰 품질 상승

AI가 만든 코드를 사람이 검토해야 하는데, 사람의 검토가 정확하다.
사람과 AI의 협업 속도가 빨라진다.

AI 성능이 올라간다.

가독성이 좋은 코드는 복잡도가 낮다. 복잡도가 낮은 코드는 AI가 실수할 확률이 줄어든다.
(Hallucination 감소)
AI는 복잡도 낮은 코드에 대해 더 정확한 수정, 예상할 수 있는 방법으로 확장, 의도에 맞는 코드를 생성한다.

클린코드를 만들기 위해서는?

AI가 생성한 코드를 리뷰하다 보면, 코드가 읽기 어렵거나 복잡하게 느껴지는 경우가 있다. 이처럼 가독성이 떨어지거나 구조적인 문제가 있다고 판단될 때, 리팩토링을 수행해야 한다.

Rule of Three (3의 법칙)을 사용하자.

성급한 판단으로 추상화를 위한 리팩토링 진행 시, 코드 복잡도가 더 올라가기에 3회째 보이게 될 때 그때 리팩토링을 진행하자는 법칙이다.

- 1번째 관찰 : 그냥 코드 작성
- 2번째 관찰 : 중복이라고 생각이 들지만 지켜보고 넘어간다.
- 3번째 관찰 : 한번 더 보이면 리팩토링을 진행한다.

리팩토링은 그 범위에 따라 접근 방식이 달라질 수 있다.

작은 리팩토링(변수명 변경, 함수 분리 등)은 즉시 수행한다.

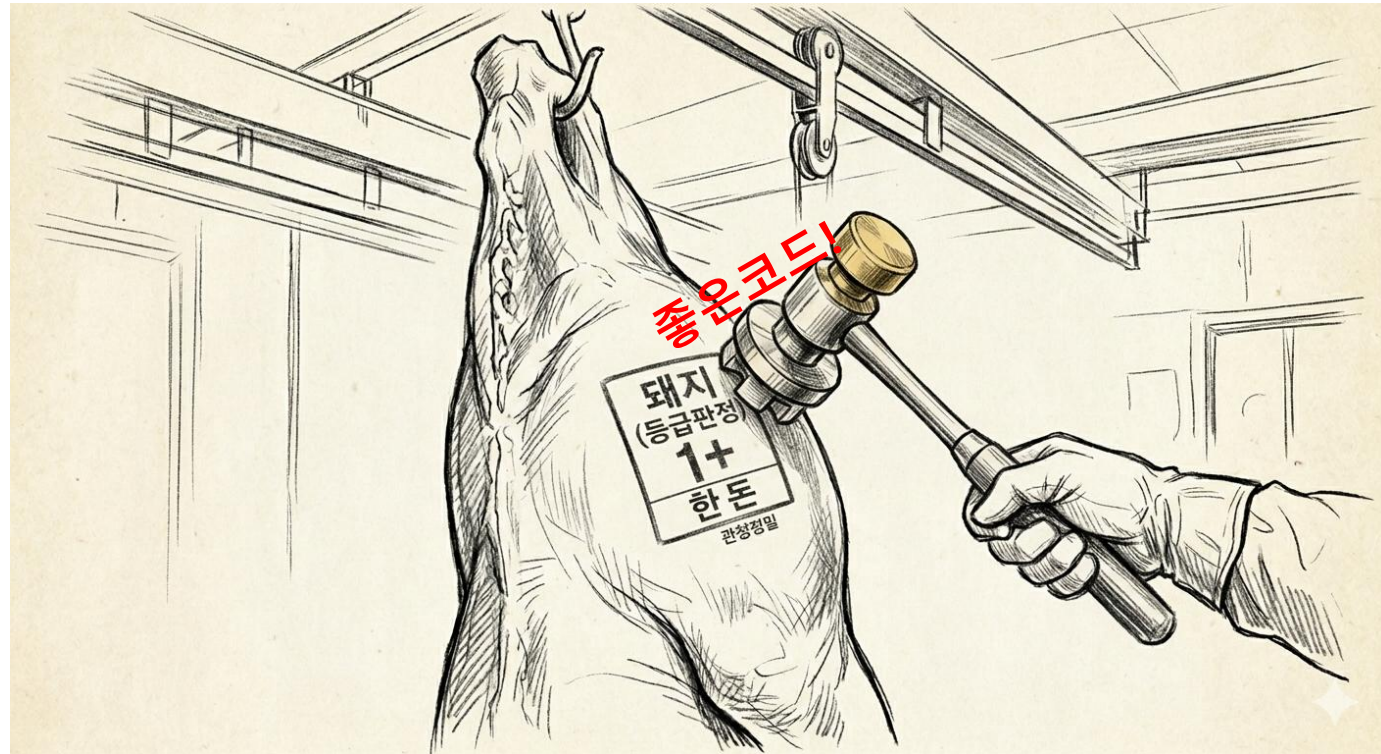
Class Level의 구조적 리팩토링이 필요한 경우에는, Agentic Engineering을 적용하여 별도의 브랜치를 생성하고 리팩토링 방향성에 대해 충분히 논의한 후 수행 및 리뷰를 진행한다.

사이먼윌리슨의 좋은코드

- 정확하게 동작한다 (버그 없이 의도대로 실행)
- 검증 가능하다 (테스트 등으로 올바름을 확인할 수 있음)
- 올바른 문제를 해결한다 (문제 정의에 부합)
- 예측 가능하게 실패한다 (에러 시 원인 파악 가능)
- 단순하고 이해 가능하다 (과도한 복잡성 없음)
- 테스트로 보호된다 (회귀 방지)
- 문서와 일치한다 (코드 변경 시 문서도 함께 갱신)
- 변경에 유연하다 (YAGNI 준수 + 변화 대응 가능)
- 비기능 품질을 갖춘다 (성능, 보안, 유지보수성, 확장성, 관찰 가능성 등)

좋은 코드 판정하기

- AI가 생성한 코드는 반드시 사람이 직접 코드 리뷰를 수행해야 한다.
- 리뷰가 완료되면, 해당 코드가 검증되었음을 나타내는 명시적인 기록을 남겨야 한다.
- 이 기록은 Commit 시 Commit Message를 통해 표현할 수 있으며, 팀에서 정의한 Commit Message Template을 활용하여 "좋은 코드 판정 마크" 의미의 기록을 남기는 것이 바람직하다.



리뷰는 책임이 따라야 한다.

- 리뷰는 책임을 가져야 한다. AI 시대 이전과 그 책임의 무게는 완전히 다르다.
- AI 시대에서 코드 리뷰는 단순한 확인 절차가 아니라, 결과에 대한 책임을 지겠다는 행위이어야 한다.
- 과거에는 개발자가 직접 코드를 작성했지만, 이제는 AI가 초안을 생성하고, 인간이 최종 검증자 역할을 수행한다.
- 따라서 개발자는 단순히 코드를 읽고 넘어가는 것이 아니라,
- 이 코드를 충분히 검토했으며, 문제가 발생할 경우 그 책임을 내가 진다 라는 의미의 판단을 기록으로 남겨야 한다.

AI가 코드를 작성하는 시대에는,
코드를 만든 주체(AI)가 아니라 승인한 사람이 책임을 진다.

[도전] 리뷰 도장 만들기

코드 책임 기반 리뷰 정책 예시

- 코드를 생성하여 Commit하는 사람은 그 코드에 책임자이며, 잘못되는 경우 불이익을 받을 수 있음을 이해한다.
- Commit에 "리뷰 도장" 내용을 남긴다.
- 코드에 문제가 발생했을 때, Commit을 수행했던 사람이 책임을 지고, 끝까지 원인을 파헤친다. 이러한 이력은 기록을 남겨야 한다.
- 리뷰 실패로 인한 이력은 누적 관리되고, 부서장은 누적된 리뷰실패 이력을 모니터링 할 수 있어야 한다.

[도전]

- Commit 메시지에 어떻게 "리뷰 도장"의 의미를 갖는
- Template을 가져갈지 샘플을 각자 만들어보자.



Chapter 08

리팩토링 실습 (차량조립 KATA)

SW 품질을 위한 Agentic Engineering

차량 제조 순서

1) 자동차 타입을 선택한다.

세단, SUV(에스-유-브이), 트럭

2) 자동차에 들어갈 부품을 선택한다.

엔진, 제동장치, 조향장치

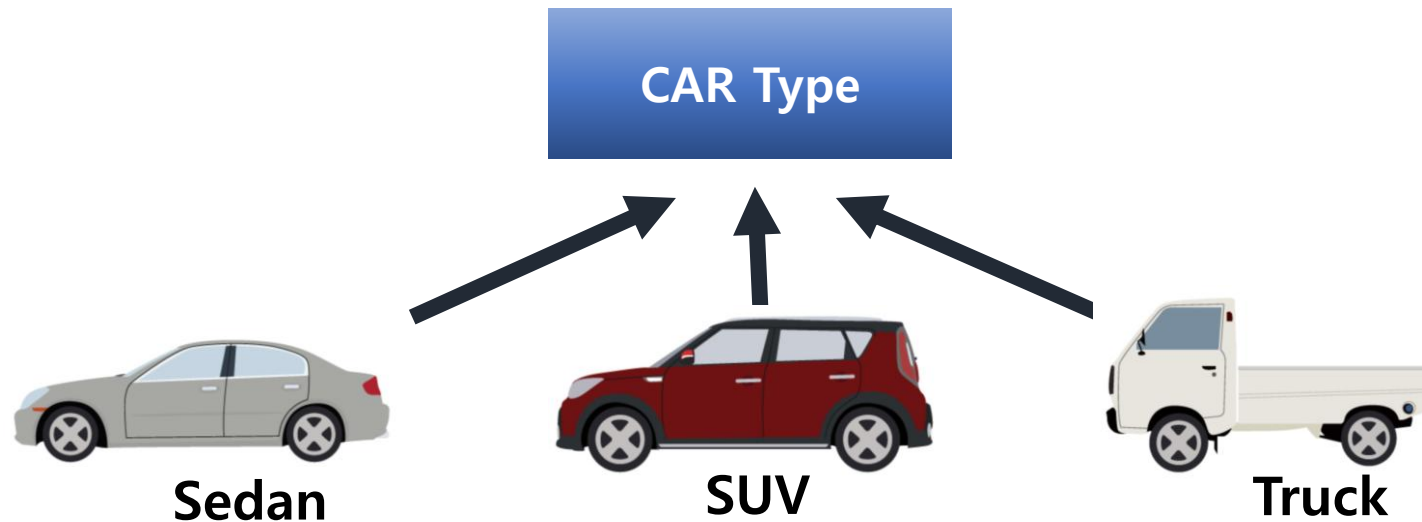
3) 완성된 차량을 테스트한다.

선택한 부품이 자동차 타입에 사용 가능한지 검사

제조 순서 1) 자동차 타입 선택

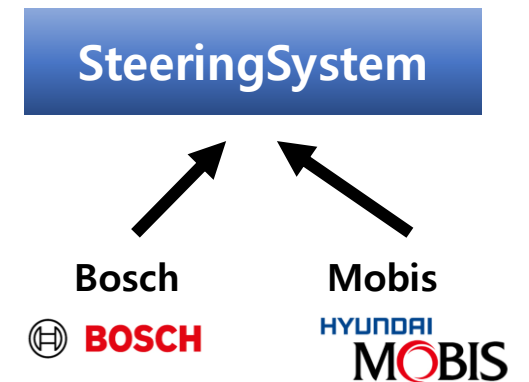
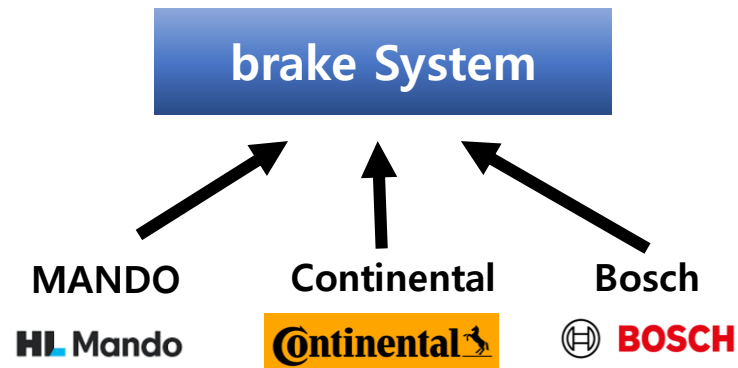
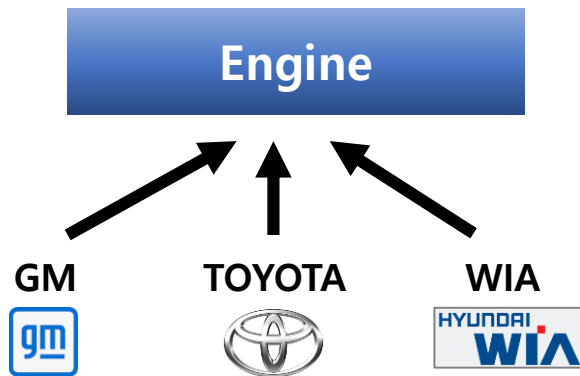
차량의 종류를 선택한다.

총 세가지 타입을 제작할 수 있으며,
향후에 타입이 더 추가될 수 있다.



제조 순서 2) 자동차 조립

엔진, 제동장치, 조향장치를 각각 선택한다.



제조 순서 3) 자동차 완성 가능조합 확인

제한조건 1

- 제동장치에 Bosch 제품을 사용했다면, 조향장치도 Bosch 제품을 사용해야한다.
(타사 제품과 호환되지 않는다.)

제한조건 2

- Continental은 Sedan용 제동장치를 만들지 않는다.
(-> 세단에 Continental 제품 사용 불가)
- 도요타는 SUV용 엔진을 만들지 않는다.
- WIA는 Truck용 엔진을 만들지 않는다.
- Mando는 Truck용 제동장치(brake System)을 만들지 않는다.

Legacy System 제공되는 소스코드

이미 제작된 소스코드가 존재한다.

절차지향식으로 구현된 소스코드

<https://github.com/mincoding-ai/assemblyCar>

기존에 사용중인 시스템의 아쉬운 점

- 절차지향식 코드로, 유지보수가 어려운 구조
- 안전하지 않은 문법들이 사용
- 확장성이 고려되지 않음
- 유닛테스트가 없음

Remote Repository

Github 에 Remote Repository를 생성

1. Owner : 실습용 Organization 선택

2. Repository 이름

- repo 이름 : **car-assembly-사번**
- Description : 소속 & 이름
- 접근 권한 : Public

Create a new repository

A repository contains all project files, including the revision history.

Owner

Repository name *

 reviewer-01 ▾

 /

car-assembly-사번

Great repository names are short and memorable. Need inspiration? How about [redesigned-journey](#)?

Description (optional)

소속 & 이름

☒  Public

Any logged in user can see this repository. You choose who can commit.

☐  Internal

Samsung SDS [enterprise members](#) can see this repository. You choose who can commit.

☐  Private

You choose who can see and commit to this repository.

Skip this step if you're importing an existing repository.

☐ Initialize this repository with a README

This will let you immediately clone the repository to your computer.

Add .gitignore: [None](#) ▾

Create repository

Chapter 09

PoC와 Collection 수집

SW 품질을 위한 Agentic Engineering

PoC(Proof of Concept)를 통해서 “이 아이디어가 실제로 작동하는가?” 를 실험해볼 수 있다.

예시

- 익숙하지 않은 기능 개발
- 구현 방식이 예상되지 않는 기술
- 적용해볼 기술의 성능 평가

현재 프로젝트에 바로 적용하기보다 PoC(Proof of Concept, 탐색적 프로토타입)을 먼저 수행하는 것이 좋다.

PoC 접근 방식

1. 현재 프로젝트와 분리된 독립적인 환경에서 진행한다
2. 빠르게 검증하는 것이 목적이므로, **Vibe Coding 방식**으로 빠르게 구현한다.
3. 이 단계에서는 Agentic Engineering을 적용할 필요는 없다.

PoC 비용

PoC 비용 절감

과거: PoC 위해 시간과 비용이 많이 들음

현재: Agent 사용으로 PoC 개발 비용이 매우 저렴하다.

→ 따라서 PoC를 적극적으로 활용하여,
단순히 논의로만 끝내는 것이 아니라
실질적인 결과인 **기술 검증과 실험**을 수행하는 것이 중요하다.

PoC가 필요한 예시 1 - 자주 사용되지 않은 Library

익숙치 않은 라이브러리 사용

- 라이브러리 코드 이해 및 사용 방법 숙달을 위해 PoC
- 코드 이해가 됐으므로 본 프로젝트에서 **코드리뷰와 함께 agentic engineering** 을 할 수 있음

PoC 예시

- 결제기능
- 구글로그인

PoC가 필요한 예시 2 – 동작 원리 이해하기

이해가지 않는 알고리즘, 동작 원리

PoC를 이용해 원리를 파악하는 코드를 작성해볼 수 있다.

예시)

- step 별로 코드 동작 원리를 이해할 수 있는 App 제작
- 동작을 시각화 해주는 App 제작
- 최소한의 이해하기 좋은 샘플코드 제작

→ 이를 통해 알고리즘의 흐름을 완전히 이해하고,
본 프로젝트에 사용하고 검토할 수 있는 기반이 될 수 있다.

[도전] PoC 만들기

익숙하지 않은 코드를 직접 PoC 직접 해보고, 코드에 대해 이해해본다.

- Json 데이터 파싱 / Json 파일 저장하는 Library - PoC 해보기
- Quick Sort 를 모른다고 가정하고, 알고리즘 PoC 해보기

Collection - 좋은 프롬프트 / 코드를 모으자.

좋은 프롬프트를 모으자

- 내가 지시할 때 사용할 우수한 프롬프트 세트를 체계적으로 정리하고 Skill로 만들어도 좋다.
- 타인이 만든 좋은 프롬프트나 Skills를 사용할 때는 반드시 내용을 이해한 후에만 사용한다.
- Skills의 동작과 목적을 모른 채로 사용하는 것은 금지한다.

좋은 예제 코드를 모으자.

내가 코드를 완벽히 이해한, 마음에 드는 좋은 코드들을 모아두자.

✓ PoC가 끝난 코드들만 모아두어도 좋다.

Collection 들을 모아두고 아래와 같은 방식으로, 지시할 수 있다.

[github.com/ ecosystem/datasette-oauth](https://github.com/ecosystem/datasette-oauth) 방식을 참고하여,
~/dev/ecosystem/llm-mistral 프로젝트에 모의 HTTP 테스트를 추가하세요.

프롬프트 / 소스코드를 모아둔 나만의 Collection을 만들자.

검증하고 이해한 자료들만 선별하여 정리

- 단순 수집이 아니라, 재사용 가능한 나만의 기준과 패턴으로 축적하자.
- 언제든지 꺼내 쓸 수 있도록 구조화된 형태로 관리하자.

모아둘 때 사용하는 도구 예시

GitHub / Gist

Confluence / Notion / Obsidian

[도전] PoC 활용하여 Application 개발

기존에 진행했던 PoC(Proof of Concept) 코드의 구조와 로직을 참고하여,
데이터를 JSON 파일로 관리하는 CRUD(Create, Read, Update, Delete) 콘솔 애플리케이션을 개발한다.

요구사항

- PoC에서 사용된 코드 구조를 유지한 상태로, CRUD 구현

CRUD 기능 구현

- **Create:** 새로운 데이터를 입력 받아 JSON 파일에 저장.
- **Read:** 전체 목록 보기 및 특정 ID/키 값으로 검색 기능.
- **Update:** 기존 데이터를 선택하여 특정 필드 수정.
- **Delete:** 특정 데이터를 안전하게 삭제.

Chapter 10

AE 관점으로 바라보기 (with SubAgent)

SW 품질을 위한 Agentic Engineering

챕터의 목표

앞으로 다양한 Agent 기능들이 추가될 예정이다.

- SubAgent는 이미 잘 활용되고 있고,
- Agent가 사람에게 먼저 질문하는 기능
- Harness 기능들 등

이러한 기능들이 추가될수록,

각 기능을 Agent Engineering 관점에서 어떻게 사용할 지
팀에서 고민하고 결정해야 한다.

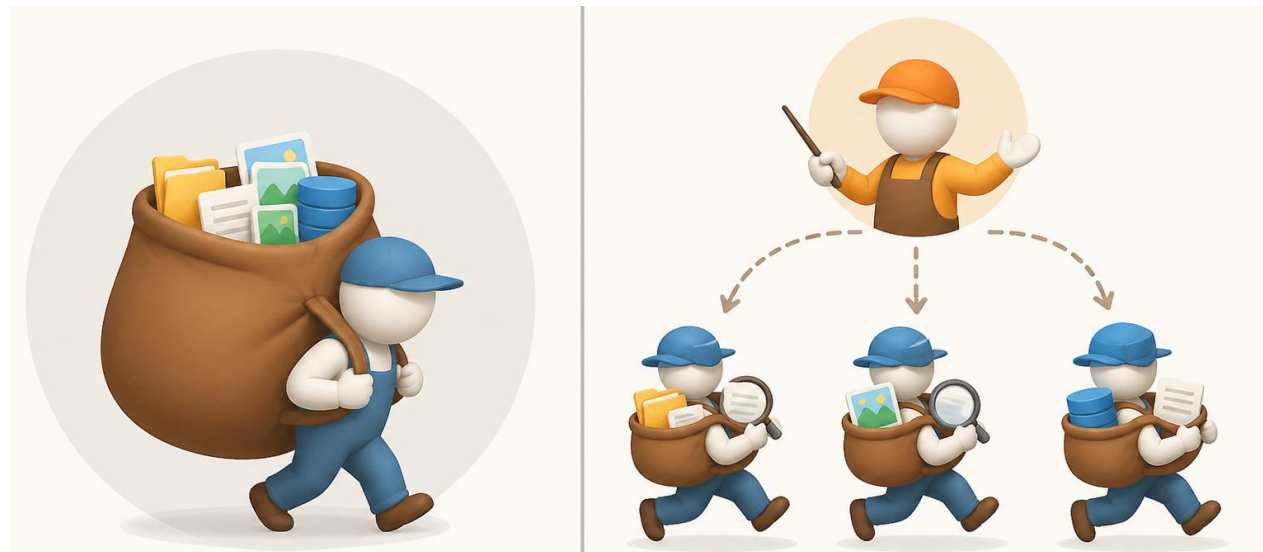
이번 챕터에서는 SubAgent 를 예시로 A.E 관점에서 바라볼 것이다.

[복습] Subagent

SubAgent는 하나의 큰 Agent 내부에서 특정 역할을 맡아 독립적으로 동작하는 “작은 Agent”이다.

- 예시 : **Main Agent (Orchestrator)**
 - SubAgent1 : Plan 작성
 - SubAgent2 : 기존 코드 분석
 - SubAgent3 : 기존 코드 테스트

이는 AI 개발 실행 시간 줄이고자 하는 목적이 아님을 기억해야 한다.
각 Agent가 본인의 역할에 맞는 **독립적인 Context Window**로 가지게 되며,
이는 더 높은 정확도를 가진 결과물을 낼 수 있는 효과가 있다.



SubAgent의 단점

SubAgent 구조는 AI의 성능을 올릴 수 있지만, 그 만큼 사람의 검증이 더 어렵다.

단일 Agent 실행

하나의 흐름으로 결과가 나오기 때문에, 결과를 실시간으로 평가가 가능하다.

SubAgent 구조

SubAgent의 중간 결과에 대해 검토하기 어렵다.

특히 병렬로 수행되는 경우, 중간 검토도 하기 전에 최종 결과를 만들어낸다.

AE 관점에서 SubAgent 사용 방안

SubAgent를 Agentic Engineering 원칙에 맞는 구조로 사용하기

Plan 이후 Action 단계

- 전체 Plan을 먼저 수립한 뒤, 실제 실행(Action)은 SubAgent에게 위임한다.
- Plan만 명확하게 유지된다면 내부 실행 방식은 유연해도 좋다.
- 각 SubAgent는 자신의 역할에 맞는 독립적인 Context로 작업을 수행하므로 결과 정확도가 올라간다.

Human-in-the-loop 이전, Verify 단계

- Action 수행 이후, 사람의 개입 전에 중간 검증 단계를 둘 수 있다.
- 이때 SubAgent를 활용하면 검증 역할을 독립적으로 수행할 수 있다.
- 결과적으로 사람은 더 정제된 결과를 확인하게 되어 검토 효율이 올라간다.

PoC (탐색적 개발) 단계

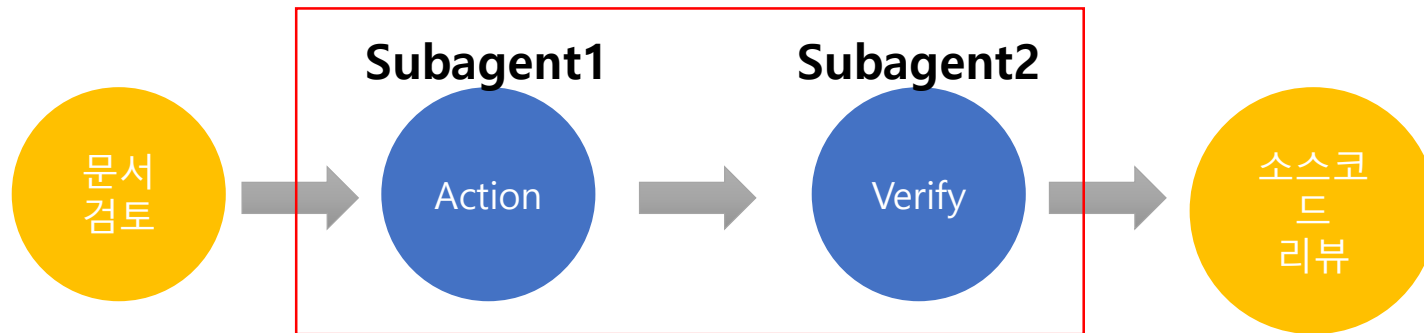
- 중간 검토가 필수적이지 않은 상황이므로, SubAgent는 자유롭게 사용 가능하다.

[도전] SubAgent를 AE 에 맞게 활용해보기

SubAgent 기반 JSON CRUD 기능 확장 및 검증

기존에 개발된 JSON-CRUD 콘솔 앱을 기반으로,
코드 구현(Action) -> 결과 검증(Verify) 과정을 각 SubAgent가 담당하여 순차적으로 완료한 후
결과를 보고한다.

Agentic Engineering에 맞는 행동이 되게끔
SubAgent를 활용해보자.



감사합니다.